

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF COMPUTING
THANJAVUR, TAMIL NADU, INDIA – 613401

Wine Quality Prediction Using a Weighted Ensemble: Reproduction, Class Balancing and Feature Importance

Report submitted to the SASTRA Deemed to be University
as a requirement for the course

CSE425 – MACHINE LEARNING ESSENTIALS

Submitted by

Surianandhan S
(Reg. No.: 127018060, B. Tech Computer Science and Business Systems)

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF COMPUTING

THANJAVUR, TAMIL NADU, INDIA – 613401

Bonafide Certificate

This is to certify that the report titled **"Wine Quality Prediction Using a Weighted Ensemble: Reproduction, Class Balancing and Feature Importance"** submitted as a requirement for the course, **CSE425: Machine Learning Essentials** for B.Tech. is a bonafide record of the work done by **Surianandhan S** (Reg. No.: 127018060, B.Tech Computer Science and Business Systems) during the academic year 2026-27, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation : Dr. Vidivelli S, Assistant Professor–II, School of Computing

Date :

Project *Viva voce* held on 2nd November 2026

Examiner 1 **Examiner 2**

Acknowledgements

I would like to thank our Honourable Chancellor **Prof. R. Sethuraman** for providing me with an opportunity and the necessary infrastructure for carrying out this project as a part of my curriculum.

I would like to thank our Honourable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of my college life.

I extend my sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

I extend my heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K. Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare.

My guide **Dr. Vidivelli S**, Assistant Professor–II, School of Computing was the driving force behind this work from the start. Her comments at the first review – to complete the class balancing, to improve the result and to study feature importance – shaped the second half of the project, and her suggestions helped me throughout. I also thank the project review panel members for their valuable comments, which made this project better.

I would like to extend my gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped me in the completion of the project.

I gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. I thank you all for providing me an opportunity to showcase my skills through this project.

List of Figures

Figure No.	Title	Page No.
2.1	Proposed architecture of the project	5
2.2	Number of wines per quality grade (the two majority grades in light colour)	7
2.3	Four measurements by quality grade	7
2.4	Pearson correlation between the 11 measurements and quality	8
4.1	Overall accuracy of the six models (the soft-vote ensemble dark, Random Forest gold)	17
4.2	Pooled confusion matrices of the Review-1 ensemble, the Review-1 Random Forest and the best Review-2 model	18
4.3	Final weights of the four learners: our reproduction and the paper's Table 8	19
4.4	Effect of the four balancing schemes on accuracy, macro-F1 and per-class recall	22
4.5	Ensemble accuracy against the temperature T (T = 1 is the paper)	23
4.6	Temperature chosen by the inner cross-validation of each training fold	23
4.7	Accuracy against macro-F1 for every candidate; the dashed line is the best trade-off frontier	24
4.8	The leading candidates on accuracy, balanced accuracy and macro-F1	26
4.9	Permutation importance on the held-out folds (macro-F1 lost when a feature is shuffled)	28
4.10	The three importance measures side by side, with the direction of each effect	29
4.11	Accuracy and macro-F1 against the number of laboratory tests used	30
5.1	Architecture of the web application	32
5.2	Overview page: the project in brief and the headline numbers	36
5.3	Dataset page: class distribution, summary statistics and correlations	37
5.4	Review 1 page: model ranking and comparison with the published benchmarks (part 1 of 2)	38
5.5	Review 1 page: model ranking and comparison with the published benchmarks (part 2 of 2)	39
5.6	Review 2 page: class balancing	39
5.7	Review 2 page: improving the result (part 1 of 2)	40
5.8	Review 2 page: improving the result (part 2 of 2)	41
5.9	Feature importance page (part 1 of 2)	42
5.10	Feature importance page (part 2 of 2)	43

Figure No.	Title	Page No.
5.11	Predict page: grades and probabilities from both models, and batch prediction	44
6.1	Output of the automated checks of the web application	50
6.2	Output: headline metrics of Review 1 and the best Review-2 model	50

List of Tables

Table No.	Title	Page No.
1.1	Summary of related works	2
1.2	Reference Paper Result: accuracy and final weight of each learner	4
1.3	Reference Paper Result: per-class results of the ensemble (%), with the earlier benchmark	4
2.1	Description of the red-wine dataset	6
2.2	Descriptive statistics of the 11 measurements (1,599 wines) and their Pearson correlation r with quality	6
2.3	Evaluation protocol	9
3.1	The four base learners of the reproduction	11
3.2	Candidate models of Review 2	15
4.1	Overall accuracy (%): Reference Paper Result vs Our Reproduction Result	17
4.2	Per-class recall (%) – the paper's "precision": benchmark, Reference Paper Result and Our Reproduction Result	18
4.3	Our Reproduction Result: conventional per-class metrics of the soft-vote ensemble (%, pooled predictions)	18
4.4	Final ensemble weights: Reference Paper Result (Table 8 of the paper) vs Our Reproduction Result (mean of 10 repeats)	19
4.5	Check (a): what the paper's own confusion matrix implies	20
4.6	Per-class values of the paper recomputed from its confusion matrix (%)	20
4.7	Check (b): the paper's weights applied to its own base accuracies	20
4.8	Class balancing applied to the paper's ensemble (%)	21
4.9	Macro-F1 of every learner under every balancing scheme	21
4.10	Weight sharpening of the Review-1 ensemble (diagnostic only)	22
4.11	Learned-temperature ensemble against the paper's weights (T = 1)	23
4.12	All Review-2 candidates ranked by macro-F1 (%, 100 fold-runs each)	25
4.13	Paired comparison over the 10 repeats	27
4.14	Feature importance: permutation importance (macro-F1 points), MDI and Spearman correlation	28
4.15	Random Forest on the top-k features chosen inside each training fold (%)	29
4.16	Reference Paper Result vs Our Reproduction Result (Review 1) vs the best Review-2 model	30
5.1	Technology stack of the project	33
5.2	Pages of the web application	33

Table No.	Title	Page No.
5.3	Example wines in the web application: three key measurements and the predicted grade (probability)	34
5.4	Automated checks of the web application	34
5.5	Structure of the GitHub repository	35

Abbreviations

Abbreviation	Meaning
BA	Balanced Accuracy (mean of the per-class recalls)
CV	Cross-Validation
ELM	Extreme Learning Machine
F1	Harmonic mean of precision and recall
FN, FP	False Negatives, False Positives
GB	Gradient Boosting
HistGB	Histogram-based Gradient Boosting
LR	Logistic Regression
MDI	Mean Decrease in Impurity
MLP	Multilayer Perceptron
OOF	Out-of-Fold (prediction made by a model that did not see the sample)
RBF	Radial Basis Function
ReLU	Rectified Linear Unit
RF	Random Forest
RNN	Recurrent Neural Network
SMOTE	Synthetic Minority Over-sampling Technique
SO_2	Sulphur dioxide
SVM	Support Vector Machine
TN, TP	True Negatives, True Positives
UCI	University of California, Irvine (Machine Learning Repository)

Notations

English Symbols

N	Number of wines (1,599)
K	Number of quality classes (6: grades 3 to 8)
M	Number of base learners in an ensemble
$x_i,\ x_{i'}$	Feature vector of wine i; its min–max scaled value
$y_i,\ \hat{y}_i$	True and predicted quality grade of wine i
$p_m(x)$	Class-probability vector given by learner m for wine x
w_m	Final ensemble weight of learner m
$w_m^{acc},\ w_m^{div}$	Accuracy-based and diversity-based weight of learner m
e_m	0/1 error vector of learner m over all wines
D_m	Diversity of learner m (sum of distances to the other error vectors)
T	Temperature that sharpens the weights (T = 1 is the paper)
n_k	Number of wines in class k
TP_k, FP_k, FN_k	True positives, false positives and false negatives of class k
$P_k, R_k, F1_k$	Precision, recall and F1 score of class k
I_j	Permutation importance of feature j
B	Number of trees in a Random Forest

Greek Symbols

ε_m	Cross-validated error rate of learner m
λ	Random interpolation factor of SMOTE, drawn from U(0, 1)
γ	Width parameter of the RBF kernel
η	Learning rate of Gradient Boosting
α_i	Dual coefficients of the SVM
ρ	Spearman rank correlation

General Terms

Classification: Assigning each sample to one of a fixed set of classes; here, a wine to one of six quality grades.

Ensemble Learning: Combining several models so that their joint prediction is more reliable than that of any one of them.

Weighted Voting: Combining the members of an ensemble with a weight per member: a soft vote adds their class probabilities, a hard vote adds their predicted labels.

Diversity: The degree to which the members of an ensemble make different errors; an ensemble only helps when its members disagree.

Class Imbalance: A large difference in the number of samples per class; here grades 5 and 6 hold 82.5 % of the wines and grades 3 and 8 only 1.75 %.

Class Weighting: Giving the samples of rare classes a larger weight in training, so that errors on them cost more.

SMOTE: Creating synthetic training samples of a rare class by interpolating between a sample and one of its nearest neighbours of the same class.

Stratified Cross-Validation: Splitting the data into k folds with the same class proportions, training on k – 1 folds and testing on the remaining one, k times.

Data Leakage: Letting information from the test data reach the training step (for example, through a scaler fitted on all the data), which gives optimistic scores.

Macro-F1: The F1 score averaged over the classes with equal weight, so that a rare class counts as much as a common one.

Balanced Accuracy: The average of the per-class recalls; a model that always predicts the majority class scores only 1/K.

Stacking: Training a second-level model (here a logistic regression) to combine the predictions of the ensemble members.

Permutation Importance: The drop in a model's score on held-out data when the values of one feature are shuffled.

Reproduction Study: Re-implementing a published method on the same data to check whether its results can be obtained independently.

Web Application: An interactive Streamlit front end that presents the results and predicts the quality of new wines in a browser.

Abstract

Wine retailers price, stock and promote wines by their quality grade, but the grade comes from a panel of tasters, while the physicochemical laboratory measurements of every batch are cheap, objective and available early. This project reproduces the weighted-ensemble framework of Ma et al. (Journal of Business Research, 2024), which predicts the tasters' grade of the 1,599 red Vinho Verde wines of the UCI Wine Quality dataset from 11 laboratory measurements and reports an overall accuracy of 89.73 %, and then extends it as requested at the first review.

The four base learners of the paper are SVM, a recurrent neural network, an extreme learning machine and Random Forest; under the classical machine-learning scope of the course the two neural models are replaced by Gradient Boosting and a multilayer perceptron. Each learner is weighted by the average of an accuracy weight (normalised inverse cross-validated error) and a diversity weight (normalised distances between error vectors), and the weighted probabilities are summed. All models are evaluated with stratified 10-fold cross-validation repeated 10 times, with min–max scaling fitted inside each training fold. The reproduced ensemble reaches 69.01 % accuracy and Random Forest alone 71.03 %. Checks computed from the paper's own tables explain the gap: its final weights are close to uniform, its base accuracies imply about 67.52 %, and its confusion matrix implies a 491-wine hold-out with 82.08 % accuracy.

Three extensions follow. Class balancing (class weights on all learners, and SMOTE inside each training fold) raises the rare-class recall: the best model, Random Forest with SMOTE, raises macro-F1 from 36.92 to 40.92 and class-4 recall from 0.57 % to 21.89 %, at a cost of 1.08 accuracy points. Learning the sharpness of the ensemble weights inside each training fold improves the paper's own ensemble to 70.89 %. Permutation importance, impurity importance and rank correlation agree that alcohol, sulphates and volatile acidity drive the prediction, and four laboratory tests already give most of the accuracy. The models are delivered through a deployed Streamlit web application.

Keywords: Wine Quality, Ensemble Learning, Weighted Voting, Reproduction Study, Class Imbalance, SMOTE, Random Forest, Permutation Importance, Streamlit

Table of Contents

Section	Title	Page No.
	Bonafide Certificate	ii
	Acknowledgements	iii
	List of Figures	iv
	List of Tables	vi
	Abbreviations	viii
	Notations	ix
	General Terms	x
	Abstract	xi
Chapter 1	**SUMMARY OF THE BASE PAPER**	**1**
1.1	Introduction	1
1.2	Related Works	2
1.3	Problem Statement	3
1.4	Objectives	3
1.5	Methodology and Results of the Base Paper	3
1.6	Scope of this Project	4
Chapter 2	**PROPOSED ARCHITECTURE AND DATA PREPROCESSING**	**5**
2.1	Proposed Architecture	5
2.2	Dataset Description	5
2.3	Exploratory Data Analysis	7
2.4	Data Preprocessing	8
2.5	Evaluation Protocol	9
2.6	Evaluation Metrics	9
Chapter 3	**MODELS**	**11**
3.1	Overview of the Base Learners	11
3.2	Support Vector Machine	11
3.3	Random Forest	11
3.4	Gradient Boosting	12
3.5	Multilayer Perceptron	12
3.6	The Weighted Ensemble of the Base Paper	12
3.7	Class Balancing Schemes	13

Section	Title	Page No.
3.8	Weight Sharpening with a Temperature	14
3.9	Learned Temperature	14
3.10	Stacking and Stronger Members	15
3.11	Feature Importance Methods	15
Chapter 4	**EVALUATION AND RESULTS**	**17**
4.1	Review 1: Model Ranking	17
4.2	Per-Class Comparison with the Base Paper	17
4.3	Learned Weights and the Paper's Weights	19
4.4	Why 89.73 % Does Not Reproduce	19
4.5	Review 2: Class Balancing	21
4.6	Review 2: Improving the Result	22
4.7	Consistency Across Repeats	26
4.8	Feature Importance	27
4.9	Reference Paper Result vs Our Result	30
4.10	Discussion	30
Chapter 5	**WEB APPLICATION: DESIGN AND DEPLOYMENT**	**32**
5.1	Purpose of the Web Application	32
5.2	System Architecture	32
5.3	Technology Stack	33
5.4	Pages and Features	33
5.5	Worked Examples	34
5.6	Verification of the Application	34
5.7	Deployment	35
5.8	Snapshots of the Web Application	36
Chapter 6	**SOURCE CODE AND SNAPSHOTS**	**45**
6.1	Settings	45
6.2	Data Loading and Scaling	45
6.3	Base Learners	45
6.4	Ensemble Weights and Weighted Vote	46
6.5	Repeated Cross-Validation	46
6.6	Class Balancing	47
6.7	Learned Temperature	47
6.8	Feature Importance	48

Section	Title	Page No.
6.9	Web Application	49
6.10	Output Snapshots	49
Chapter 7	**MERITS AND DEMERITS**	**51**
7.1	Merits	51
7.2	Demerits	51
Chapter 8	**CONCLUSION AND FUTURE PLANS**	**52**
Chapter 9	**REFERENCES**	**53**

CHAPTER 1
SUMMARY OF THE BASE PAPER

Paper Title: From vineyard to table: Uncovering wine quality for sales management through machine learning

Authors: Rui Ma, Di Mao, Dongmei Cao, Shuai Luo, Suraksha Gupta and Yichuan Wang (University of Greenwich, Nottingham Trent University, State Grid Tianjin Electric Power Company, Newcastle University and University of Sheffield)

Journal Name: Journal of Business Research

Publisher: Elsevier

Year of Publication: 2024

Volume and Article: Volume 176, Article 114576 (15 pages)

DOI: https://doi.org/10.1016/j.jbusres.2024.114576

Indexing Status: The Journal of Business Research is a peer-reviewed Elsevier journal indexed in Scopus and in the Social Sciences Citation Index (Clarivate Web of Science).

1.1 INTRODUCTION

A wine retailer decides which wines to buy, how to price them and how to present them largely by their quality, but the quality grade is assigned by a panel of trained tasters. Sensory assessment is slow, costly and partly subjective. The physicochemical tests made on every batch – acidity, residual sugar, sulphur dioxide, density, alcohol and so on – are cheap, objective and available before the wine reaches the shelf. If the tasters' grade can be predicted from these tests, a retailer gains an early and consistent estimate of quality that can support purchasing, pricing and promotion.

The base paper [21] builds such a predictor for the 1,599 red Vinho Verde wines of the UCI Wine Quality dataset [6]. It trains four different learners – a support vector machine (SVM), a recurrent neural network (RNN), an extreme learning machine (ELM) and a Random Forest (RF) – and combines them in a weighted vote in which each learner's weight rewards both its accuracy and how differently it errs from the others. The paper reports an overall accuracy of **89.73 %**, 9.02 points above its best single learner, and identifies alcohol, sulphates, volatile acidity and citric acid as the attributes that matter most.

This project reproduces that framework within the classical machine-learning scope of the course, extends it in response to the comments of the first review – class balancing, an improved result and feature importance – and delivers the models through a web application. Throughout the report, values published in the paper are labelled **Reference Paper Result** and values obtained here **Our Reproduction Result**, so that the two are never mixed.

1.2 RELATED WORKS

Cortez et al. [5] collected the Vinho Verde data used here [6] and modelled the quality grade from the physicochemical tests with an SVM, a neural network and multiple regression, finding the SVM the most accurate and using a sensitivity analysis to rank the inputs. Later studies used the same or similar data with other learners: Gupta [13] first selected the important features with linear regression and then predicted quality with neural networks and SVMs; Dahal et al. [7] compared ridge regression, an SVM, gradient boosting and a neural network on red wine and found gradient boosting the most accurate; and Bhardwaj et al. [1] predicted the quality of New Zealand Pinot noir wines from chemical measurements, enlarging a very small sample with synthetic data before comparing several classifiers.

The base paper's idea of combining learners rests on the ensemble literature. Dietterich [9] explains why an ensemble can outperform its members, and Zhou [33] surveys combination rules including weighted voting and stacking [32]. Kuncheva and Whitaker [19] compared many measures of diversity between classifiers and found no simple link between diversity and ensemble accuracy – a point that becomes important in Chapter 4. The learners come from Cortes and Vapnik [4] (SVM), Breiman [2] (Random Forest), Friedman [11] (Gradient Boosting), Rumelhart et al. [25] (back-propagation networks) and Huang et al. [16] (ELM). For the second review, He and Garcia [15] review learning from imbalanced data, Chawla et al. [3] introduced SMOTE, Strobl et al. [28] showed that impurity-based importance is biased, and Fisher et al. [10] give the theory behind permutation importance. Table 1.1 summarises the studies closest to this project.

Table 1.1: Summary of related works

Author and year	Contribution / finding	Methods
Ma et al. (2024) [21] – base paper	Weighted ensemble for red-wine quality; reports 89.73 % accuracy, 9.02 points above its best learner	SVM, RNN, ELM, RF; accuracy- and diversity-weighted vote
Cortez et al. (2009) [5]	Introduced the Vinho Verde data; predicted quality from 11 laboratory tests and ranked the inputs	SVM, neural network, multiple regression
Gupta (2018) [13]	Selected the important physicochemical features before predicting quality	Linear regression (selection); neural network, SVM
Dahal et al. (2021) [7]	Compared four learners for red-wine quality; gradient boosting was the most accurate	Ridge regression, SVM, gradient boosting, neural network
Bhardwaj et al. (2022) [1]	Predicted Pinot noir quality from chemical data, enlarging a small sample with synthetic data	Several classifiers; synthetic oversampling
Kuncheva and Whitaker (2003) [19]	Studied diversity measures for classifier ensembles; no simple link between diversity and accuracy	Pairwise and non-pairwise diversity measures
Chawla et al. (2002) [3]	Oversampled minority classes with synthetic samples interpolated between neighbours	SMOTE

1.3 PROBLEM STATEMENT

Given the 11 physicochemical measurements of a red wine, the problem is to predict the quality grade assigned by the tasters. The grades are heavily imbalanced – grades 5 and 6 hold 82.5 % of the wines and grades 3 and 8 only 1.75 % – so a model can reach a high accuracy while never recognising the very poor or the excellent wines, which are exactly the ones a retailer most needs to find. The project must therefore (i) check whether the weighted ensemble of the base paper really reaches the reported accuracy when it is evaluated without information leaking from the test folds, (ii) improve the prediction of the rare grades, and (iii) explain which measurements drive the prediction.

1.4 OBJECTIVES

1. To reproduce the four base learners and the accuracy- and diversity-weighted ensemble of the base paper on the UCI red-wine data.
2. To evaluate every model with stratified 10-fold cross-validation repeated 10 times, with all preprocessing fitted inside the training folds.
3. To compare the reproduction with the published results and to check the paper's figures against its own tables.
4. To balance the classes with class weights and with SMOTE, and to measure the effect on every class.
5. To improve the result by sharpening the ensemble weights, by stacking and by stronger members, without using the test folds for any choice.
6. To measure feature importance with three methods and to find how many laboratory tests are needed.
7. To deliver the models through a tested, deployed web application.

1.5 METHODOLOGY AND RESULTS OF THE BASE PAPER

The paper min–max normalises the 11 attributes and trains four base learners – SVM, RNN, ELM and RF – each evaluated by 10-fold cross-validation repeated over 100 experiments. Each learner then receives two weights. The accuracy weight is its inverse cross-validated error, normalised over the learners. The diversity weight is the sum of the Euclidean distances between its 0/1 error vector and those of the other learners, also normalised. The final weight is the average of the two, and the ensemble prediction is the weighted sum of the learners' outputs. The paper reports per-class results for the six grades present (3 to 8), compares them with Cortez et al. [5], and studies how the attributes relate to quality. Tables 1.2 and 1.3 list the published values this project compares against.

The main contributions of the paper are:

1. a framework that turns product attributes into a quality prediction a retailer can act on;
2. an ensemble that weights heterogeneous learners by both accuracy and diversity, reported to beat each learner and earlier studies; and

3. an analysis of the attributes – alcohol, sulphates, volatile acidity and citric acid – most associated with higher quality.

Table 1.2: Reference Paper Result: accuracy and final weight of each learner

Learner	Overall accuracy (%)	Final weight w_i (paper's Table 8)
SVM	67.30	0.2516
RNN	61.43	0.2286
ELM	55.77	0.2168
Random Forest	80.71	0.3030
Weighted ensemble	**89.73**	–

Table 1.3: Reference Paper Result: per-class results of the ensemble (%), with the earlier benchmark

Quality grade	Wines	Cortez et al. (2009)	Ma et al. (2024)
3	10	0.00	33.33
4	53	20.00	63.16
5	681	67.50	84.80
6	638	57.70	88.06
7	199	58.60	77.08
8	18	0.00	50.00
Overall	**1,599**	**–**	**89.73**

1.6 SCOPE OF THIS PROJECT

- **Dataset:** the red-wine part of the UCI Wine Quality data (1,599 wines), as in the paper; all six grades are kept.
- **Learners:** SVM and Random Forest as in the paper; Gradient Boosting replaces the RNN and a multilayer perceptron replaces the ELM, because recurrent networks and extreme learning machines are outside the classical machine-learning scope of the course.
- **Evaluation:** stratified 10-fold cross-validation × 10 repeats (100 fold-runs), with scaling inside each training fold.
- **Review 2:** class balancing, improvement of the result and feature importance, each evaluated with the same protocol.
- **Deployment:** a Streamlit web application, run locally or deployed on Streamlit Community Cloud from the GitHub repository.

CHAPTER 2
PROPOSED ARCHITECTURE AND DATA PREPROCESSING

2.1 PROPOSED ARCHITECTURE

Figure 2.1 shows the complete workflow. Part A prepares the data and fixes the evaluation protocol. Part B reproduces the base paper: four base learners, the accuracy and diversity weights, the weighted vote and checks of the paper's own figures (Review 1). Part C makes the three changes requested at the first review: class balancing, a better result and feature importance (Review 2). Part D evaluates every model in the same way and delivers the results through the report and a web application. Every step that involves randomness uses a fixed seed, so all results are reproducible.

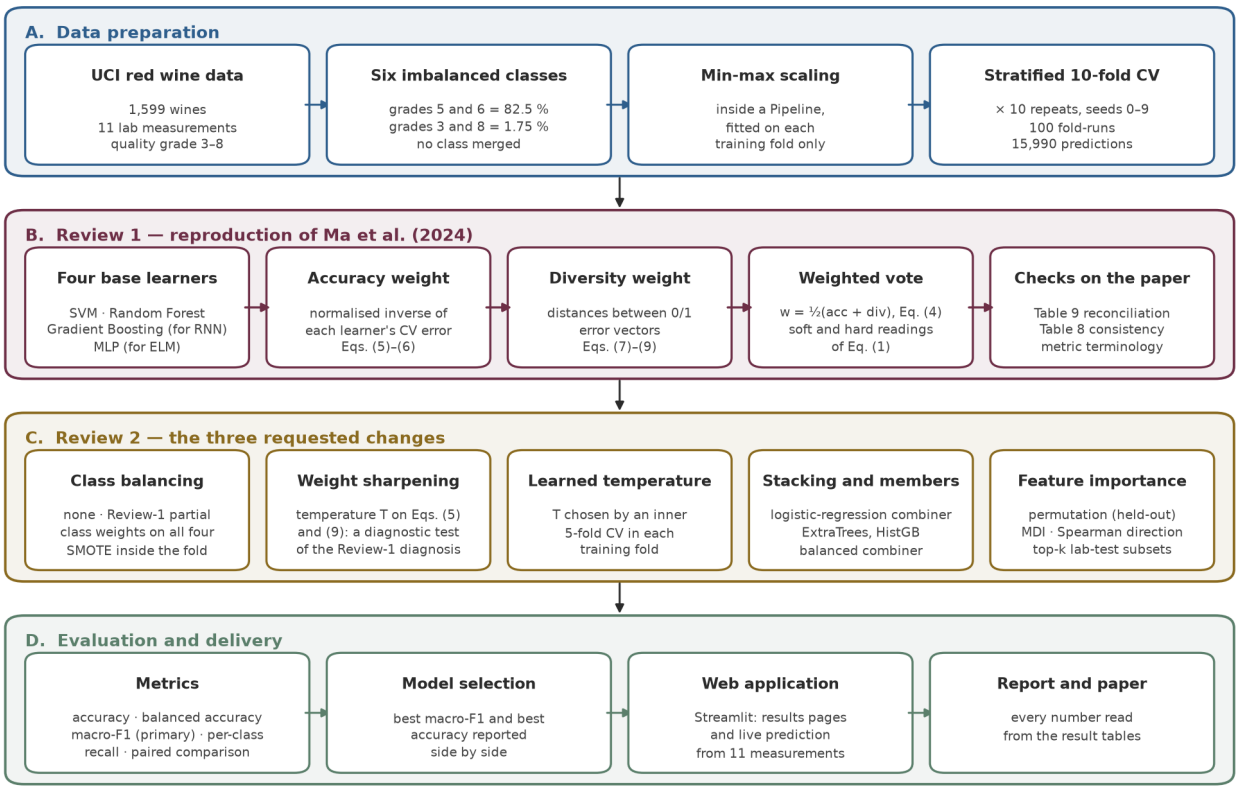


Figure 2.1: Proposed architecture of the project

2.2 DATASET DESCRIPTION

The dataset is the red-wine part of the Wine Quality data of the UCI Machine Learning Repository [6], collected by Cortez et al. [5]. Each row is one wine with 11 physicochemical measurements and the quality grade given by sensory assessors (Table 2.1).

Table 2.1: Description of the red-wine dataset

Attribute	Value
Source	UCI Machine Learning Repository [6]
Wines	Red Vinho Verde wines from the north-west of Portugal
Size	1,599 rows × 12 columns (11 inputs and the quality grade)
Inputs	fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free and total sulphur dioxide, density, pH, sulphates, alcohol
Target	quality on a 0–10 scale, the median of at least three sensory assessments; only grades 3–8 occur
Missing values	none
Duplicates	240 rows repeat an earlier row exactly; all rows are kept, as in the base paper

Table 2.2 summarises the 11 measurements. Their ranges differ by four orders of magnitude – density varies in the third decimal place, total sulphur dioxide up to 289 mg/dm³ – which is why the features are scaled before training.

Table 2.2: Descriptive statistics of the 11 measurements (1,599 wines) and their Pearson correlation r with quality

Feature	Unit	Min	Max	Mean	Std. dev.	r with quality
fixed acidity	g(tartaric acid)/dm³	4.60	15.90	8.32	1.74	+0.124
volatile acidity	g(acetic acid)/dm³	0.120	1.580	0.528	0.179	-0.391
citric acid	g/dm³	0.000	1.000	0.271	0.195	+0.226
residual sugar	g/dm³	0.90	15.50	2.54	1.41	+0.014
chlorides	g(sodium chloride)/dm³	0.012	0.611	0.087	0.047	-0.129
free sulfur dioxide	mg/dm³	1.00	72.00	15.87	10.46	-0.051
total sulfur dioxide	mg/dm³	6.00	289.00	46.47	32.90	-0.185
density	g/cm³	0.9901	1.0037	0.9967	0.0019	-0.175
pH	–	2.74	4.01	3.31	0.15	-0.058
sulphates	g(potassium sulphate)/dm³	0.33	2.00	0.66	0.17	+0.251
alcohol	% vol.	8.40	14.90	10.42	1.07	+0.476

The six grades are strongly imbalanced (Figure 2.2). Grades 5 and 6 hold 681 and 638 wines – 82.5 % together – while grade 3 has only 10 wines, grade 8 18 and grade 4 53. In a 10-fold split a test fold therefore holds about one wine of grade 3 and two of grade 8, which is the main difficulty of the task.

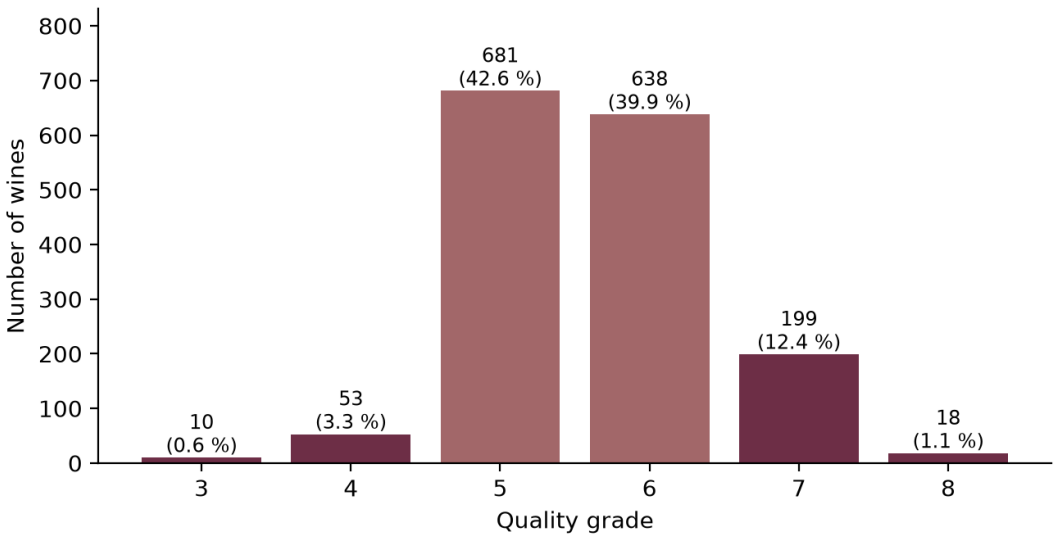


Figure 2.2: Number of wines per quality grade (the two majority grades in light colour)

2.3 EXPLORATORY DATA ANALYSIS

Figure 2.3 shows how four measurements change with the grade. Alcohol rises clearly from grade 5 upwards, volatile acidity falls, and sulphates and citric acid rise; the boxes of neighbouring grades overlap widely, however, so no single measurement separates the grades. Figure 2.4 gives all pairwise correlations. Alcohol has the strongest correlation with quality (r = +0.48), followed by volatile acidity (-0.39), sulphates (+0.25) and citric acid (+0.23) – the four attributes the base paper highlights. Several inputs are correlated with each other (fixed acidity with citric acid, density and pH; free with total sulphur dioxide), which matters later when importance is measured by shuffling one feature at a time.

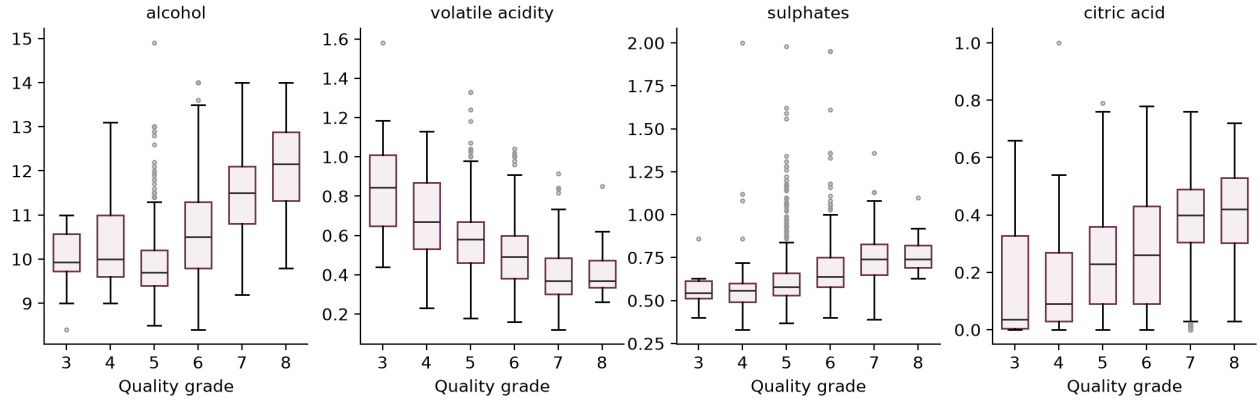


Figure 2.3: Four measurements by quality grade

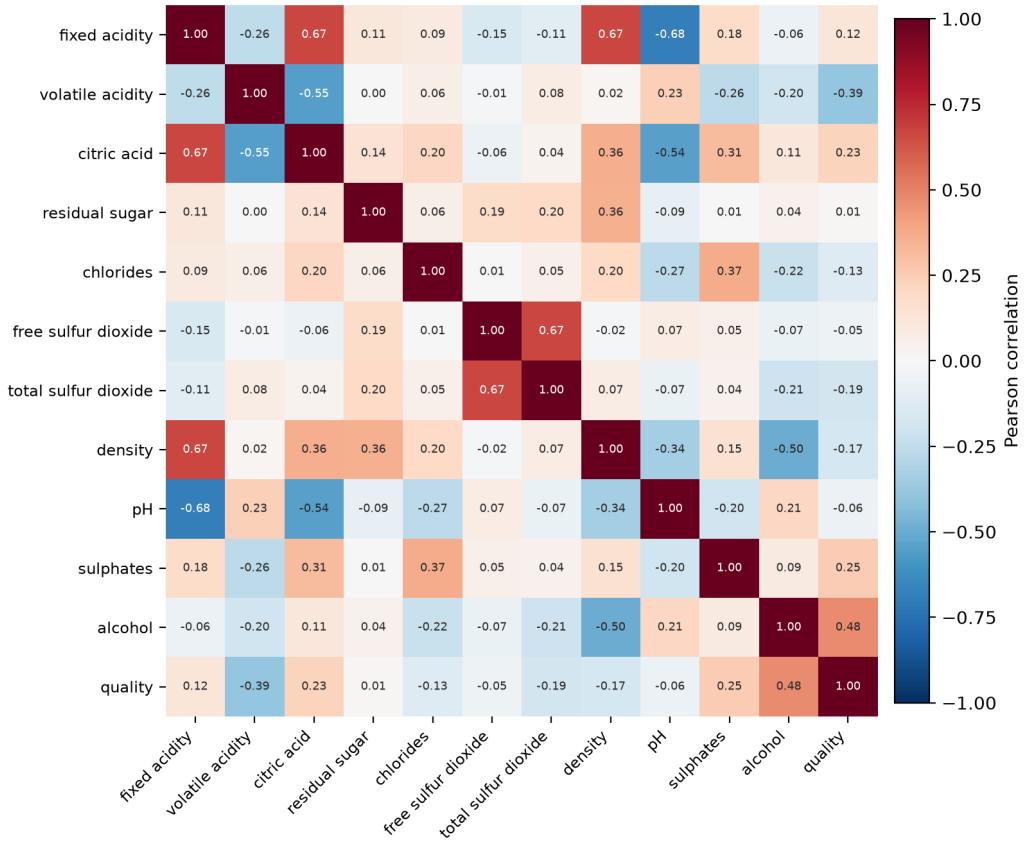


Figure 2.4: Pearson correlation between the 11 measurements and quality

2.4 DATA PREPROCESSING

2.4.1 Classes and Duplicate Rows

No grade is dropped or merged: the task keeps all six grades, exactly as the paper's tables do (grades 0–2, 9 and 10 never occur). The 240 exact duplicate rows are kept as well, because the paper uses all 1,599 wines; a duplicate can fall into a training and a test fold at the same time, which slightly favours every model equally and does not change the comparison between them.

2.4.2 Min–Max Normalisation

Following the paper, every feature is rescaled linearly to [0, 1] with (2.1), so that measurements in different units contribute comparably to the distance-based SVM and to the neural network.

$$x_{ij'} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}} \tag{2.1}$$

2.4.3 Preventing Data Leakage

If the minimum and maximum in (2.1) are computed on all the data before cross-validation, the test folds influence the scaling of the training data. In this project the scaler is the first step of a scikit-learn Pipeline [24] together with the learner, so in every fold it is fitted on the training wines only and merely applied to the test wines. The same rule is used in Review 2 for SMOTE, for feature

selection and for every choice of a setting: anything learned from data is learned inside the training fold.

2.5 EVALUATION PROTOCOL

The paper evaluates its models with 10-fold cross-validation over 100 experiments. Here this is implemented as stratified 10-fold cross-validation repeated 10 times with different shuffles (seeds 0–9): every wine is predicted once per repeat by a model that did not see it, giving 10 × 1,599 = 15,990 out-of-fold predictions per model. The metrics are computed on these pooled predictions. All learners are scored on the same folds in each repeat, which the diversity weight of the paper needs (it compares error vectors wine by wine) and which allows paired comparisons between models (Section 4.7). Table 2.3 lists the settings.

Table 2.3: Evaluation protocol

Setting	Value
Cross-validation	stratified 10-fold, repeated 10 times (fold seeds 0–9) = 100 fold-runs
Predictions per model	15,990 out-of-fold predictions (each wine 10 times)
Preprocessing	min–max scaling (and SMOTE, feature selection) fitted inside each training fold
Model seeds	random_state = 42 for every learner
Primary metric, Review 1	overall accuracy, as in the paper
Primary metric, Review 2	macro-F1, fixed in the code before any Review-2 experiment was run

2.6 EVALUATION METRICS

For class k, TP_k counts the wines of class k predicted as k, FP_k the wines of other classes predicted as k and FN_k the wines of class k predicted as something else [26]. The metrics are defined in (2.2)–(2.7).

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} 1\left[\hat{y}_i = y_i\right] \tag{2.2}$$

$$P_k = \frac{TP_k}{TP_k + FP_k} \tag{2.3}$$

$$R_k = \frac{TP_k}{TP_k + FN_k} \tag{2.4}$$

$$F1_k = \frac{2\,P_k R_k}{P_k + R_k} \tag{2.5}$$

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F1_k \tag{2.6}$$

$$\text{Balanced accuracy} = \frac{1}{K}\sum_{k=1}^{K} R_k \tag{2.7}$$

Accuracy (2.2) is dominated by grades 5 and 6. Macro-F1 (2.6) and balanced accuracy (2.7) give every grade the same weight, so they show whether the rare grades are recognised at all; this is why macro-F1 is the primary metric of Review 2.

A note on terminology. The paper calls its per-class metric "precision", but its own worked example divides the correct predictions of a class by the number of wines that truly belong to it – that is TP/(TP + FN), the recall (2.4). Recomputing the paper's per-class values from its confusion matrix confirms this: 4 of the 6 values reproduce when divided by the true-class totals and only one when divided by the predicted totals. The per-class comparisons with the paper in this report therefore use recall, under the paper's definition; conventional precision is reported separately.

CHAPTER 3
MODELS

3.1 OVERVIEW OF THE BASE LEARNERS

Table 3.1 lists the four base learners. SVM and Random Forest are those of the paper. The RNN and the ELM are neural models outside the classical machine-learning scope of the course, so each is replaced by the closest classical learner: Gradient Boosting, which, like a recurrent model, refines its prediction step by step, and a multilayer perceptron, which is a neural network like the ELM but trained by back-propagation. Every learner is a Pipeline of a min–max scaler and the classifier, and every learner outputs class probabilities, which the soft vote of the ensemble needs.

Table 3.1: The four base learners of the reproduction

Paper's learner	Our learner	Main settings	Reason
SVM	SVC (RBF kernel)	C = 10, γ = 'scale', balanced class weights, Platt-calibrated probabilities	as in the paper
Random Forest	RandomForestClassifier	300 trees, balanced class weights per bootstrap sample	as in the paper
RNN	GradientBoostingClassifier	200 boosting stages, depth-3 trees, learning rate 0.1	RNN is outside the course scope
ELM	MLPClassifier	hidden layers (64, 32), ReLU, Adam, up to 2,000 iterations	ELM is outside the course scope

3.2 SUPPORT VECTOR MACHINE

An SVM [4] separates two classes with the hyperplane of maximum margin in a feature space defined by a kernel. Its decision function (3.1) is a weighted sum of kernel values between the wine and the support vectors, with the RBF kernel (3.2). The six grades are handled one-against-one. Because the ensemble needs probabilities, the SVM is wrapped in a calibration step (Platt scaling) that is fitted by an internal cross-validation on the training fold only.

$$f(x) = \operatorname{sign}\left(\sum_i \alpha_i y_i K(x_i, x) + b \right) \tag{3.1}$$

$$K(x, x') = \exp\left(-\gamma \parallel x - x' \parallel^2 \right) \tag{3.2}$$

3.3 RANDOM FOREST

A Random Forest [2] grows B decision trees, each on a bootstrap sample of the training wines and with a random subset of features tried at every split. The class probabilities are the average of the trees' probabilities (3.3). With B = 300 and balanced class weights recomputed on every bootstrap sample, it is the strongest single learner of the reproduction.

$$\hat{p}(k \mid x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b(k \mid x) \tag{3.3}$$

3.4 GRADIENT BOOSTING

Gradient Boosting [11] builds an additive model stage by stage: each new set of small trees h_m is fitted to the negative gradient of the loss of the current model and added with a learning rate η (3.4). It stands in for the paper's RNN because both improve their prediction iteratively; it is not a sequence model, which this tabular task does not need.

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x) \tag{3.4}$$

3.5 MULTILAYER PERCEPTRON

The multilayer perceptron has two hidden layers of 64 and 32 ReLU units and a softmax output (3.5), trained by back-propagation [25] with the Adam optimiser. The ELM of the paper [16] has the same feed-forward shape but fixes random hidden weights and solves only the output layer in closed form; the MLP is the closest classical alternative. Early stopping is not used, because it would set aside a validation slice from a training fold that contains only nine grade-3 wines.

$$\hat{p}(x) = \operatorname{softmax}\Big(W_3 \operatorname{ReLU}\Big(W_2 \operatorname{ReLU}\big(W_1 x' + b_1 \big) + b_2 \Big) + b_3 \Big) \tag{3.5}$$

3.6 THE WEIGHTED ENSEMBLE OF THE BASE PAPER

The ensemble of the paper combines the M = 4 learners with weights that reward accuracy and diversity. Written in this report's notation, the steps are (3.6)–(3.13); the paper's own equation number is given in brackets. The weights are computed in each repeat from the out-of-fold predictions of that repeat, so they use exactly the information the paper describes.

Combination (paper's Eq. 1). The ensemble output is the weighted sum of the learners' outputs (3.6). Read literally, the outputs are class-probability vectors, giving a soft vote; read as one-hot labels, it becomes a weighted majority (hard) vote (3.7). The paper does not say which reading it uses, so both are implemented; the soft vote is the headline.

$$\hat{p}(x) = \sum_{m=1}^{M} w_m p_m(x), \quad \hat{y} = \operatorname*{argmax}_k \hat{p}_k(x) \tag{3.6}$$

$$\hat{y} = \operatorname*{argmax}_k \sum_{m=1}^{M} w_m \, 1\left[\hat{y}^{(m)} = k \right] \tag{3.7}$$

Accuracy weight (paper's Eqs. 3, 5 and 6). An indicator marks each wrong prediction (3.8), the cross-validated error of a learner is the mean of its indicators (3.9), and the accuracy weight is the normalised inverse error (3.10).

$$g\left(\hat{y}_i^{(m)}, y_i\right) = \begin{cases} 0, & \hat{y}_i^{(m)} = y_i \\ 1, & \text{otherwise} \end{cases} \tag{3.8}$$

$$\varepsilon_m = \frac{1}{N} \sum_{i=1}^{N} g\left(\hat{y}_i^{(m)}, y_i\right) \tag{3.9}$$

$$w_m^{acc} = \frac{1/\varepsilon_m}{\sum_{j=1}^{M} 1/\varepsilon_j} \tag{3.10}$$

Diversity weight (paper's Eqs. 7–9). The indicators of a learner over all wines form its error vector (3.11). Its diversity is the sum of the Euclidean distances to the error vectors of the other learners (3.12), normalised in (3.13).

$$e_m = \left(g\left(\hat{y}_1^{(m)}, y_1\right), \ldots, g\left(\hat{y}_N^{(m)}, y_N\right) \right) \tag{3.11}$$

$$D_m = \sum_{j \neq m} \| e_m - e_j \|_2 \tag{3.12}$$

$$w_m^{div} = \frac{D_m}{\sum_{j=1}^{M} D_j} \tag{3.13}$$

Final weight (paper's Eq. 4). The final weight is the average of the two (3.14); the weights sum to one.

$$w_m = \frac{1}{2}\left(w_m^{acc} + w_m^{div}\right) \tag{3.14}$$

A property of (3.10) and (3.13) is worth stating now, because it explains the results of Chapter 4. Both normalise quantities that differ little between reasonable learners: error rates of 29 % and 38 % give inverse errors of 3.4 and 2.6, and the distances between error vectors are of similar size. The weights therefore stay close to the uniform 1/M = 0.25, and the ensemble behaves almost like a plain average of its members.

3.7 CLASS BALANCING SCHEMES

Review 1 already used balanced class weights in SVM and Random Forest, but not in the other two learners. Review 2 compares four schemes on the same four learners and the same weighted vote, so that balancing is the only thing that changes [15]:

1. **No balancing** – every wine has the same weight.
2. **Review 1 (partial)** – balanced class weights in SVM and Random Forest only.

3. **Class weights, all four** – balanced weights in every learner. Gradient Boosting and the MLP have no class-weight option, so they receive the equivalent per-sample weights (3.15) through a small wrapper, computed from the training fold.
4. **SMOTE** [3] – synthetic wines of the rare grades are created inside each training fold, after scaling, by interpolating between a wine and one of its five nearest neighbours of the same grade (3.16), until every grade is as large as the largest. The test fold is never resampled. The imbalanced-learn library [20] provides the implementation.

$$s_i = \frac{N}{K n_{y_i}} \tag{3.15}$$

$$x_{new} = x_i + \lambda\left(x_{nn} - x_i\right), \quad \lambda \sim U(0,1) \tag{3.16}$$

3.8 WEIGHT SHARPENING WITH A TEMPERATURE

To test the explanation that the weights are too flat, a temperature T is introduced into (3.10) and (3.13): the inverse errors and the diversities are raised to the power T before normalising, (3.17) and (3.18), and (3.14) is unchanged. T = 1 is exactly the paper; a larger T moves weight towards the most accurate (or most diverse) learner. Two variants are tried: sharpening only the accuracy term, and sharpening both. This sweep is a diagnostic only: its weights are computed from the same out-of-fold predictions on which it is scored, so it is never used to select a model.

$$w_m^{acc}(T) = \frac{\varepsilon_m^{-T}}{\sum_{j=1}^{M} \varepsilon_j^{-T}} \tag{3.17}$$

$$w_m^{div}(T) = \frac{D_m^{T}}{\sum_{j=1}^{M} D_j^{T}} \tag{3.18}$$

3.9 LEARNED TEMPERATURE

The proper way to use the temperature is to choose it without the test fold. The learned-temperature ensemble does this inside every outer training fold:

1. split the training fold into 5 inner folds and collect out-of-fold probabilities of the four learners;
2. for every T ∈ {1, 2, 5, 10, 25, 100} and both variants, compute the weights (3.17)–(3.18) and score the weighted vote on the inner predictions with macro-F1;
3. keep the best setting (ties go to the smaller T, that is, towards the paper);
4. refit the four learners on the whole training fold and predict the outer test fold with the chosen weights.

The outer test fold therefore never influences the choice of T, and the result is an honest estimate.

3.10 STACKING AND STRONGER MEMBERS

Two further ideas are tested. **Stacking** [32] replaces the hand-built weights with a learned combiner: a multinomial logistic regression is trained on the members' out-of-fold class probabilities (3.19), using an inner 5-fold split. A second version gives the combiner balanced class weights. **Stronger members** replace the two weakest learners by ExtraTrees [12] (500 extremely randomised trees) and histogram-based Gradient Boosting, a fast gradient-boosting variant in the style of LightGBM [18], both class-weighted. Table 3.2 lists all candidates of Review 2.

$$\hat{p}(x) = \mathrm{softmax}\Big(\beta_0 + B\big[p_1(x);\, p_2(x);\, \ldots;\, p_M(x)\big]\Big) \tag{3.19}$$

Table 3.2: Candidate models of Review 2

Group	Candidates	What is learned inside the training fold
A. Balancing	4 learners and the paper's ensemble × {none, Review 1, class weights, SMOTE}	scaler, SMOTE samples, weights (3.14)
B1. Sharpening	paper's ensemble with T = 1 … 100 (diagnostic only)	– (weights from the scored predictions)
B2. Learned T	learned-temperature ensemble with Review-1 members and with SMOTE members	T and variant by inner 5-fold CV
B3. Stacking	logistic-regression combiner on the 4 learners; balanced combiner; on SVM + RF + ExtraTrees + HistGB	combiner by inner 5-fold CV
B3. Members	ExtraTrees, HistGB, and the paper's weights on SVM + RF + ExtraTrees + HistGB	scaler, trees

3.11 FEATURE IMPORTANCE METHODS

Three measures are computed and compared, because each has known weaknesses:

- **Permutation importance** (primary measure) [2], [10] – the drop in macro-F1 on the held-out fold when one feature is shuffled (3.20), averaged over 20 shuffles and the 10 folds. It measures what the model actually uses on unseen wines, but it shares the credit between correlated features.
- **Mean decrease in impurity (MDI)** of the Random Forest – how much the splits on a feature reduce impurity. It is free but biased towards continuous features with many distinct values [28].
- **Spearman rank correlation** (3.21) of each feature with quality – the direction of the effect (higher alcohol, better wine), which the other two measures do not give.

$$I_j = S\big(f, X_{test}\big) - \frac{1}{R}\sum_{r=1}^{R} S\Big(f, X_{test}^{(j,r)}\Big) \tag{3.20}$$

$$\rho = 1 - \frac{6\sum_{i=1}^{N} d_i^2}{N\big(N^2-1\big)} \tag{3.21}$$

Finally, to find how many laboratory tests are needed, Random Forest is retrained on the top-k features for k = 1, 2, 3, 4, 6, 8 and 11. The ranking used to choose the features is recomputed inside every training fold, so the test fold never influences which features are kept.

CHAPTER 4
EVALUATION AND RESULTS

4.1 REVIEW 1: MODEL RANKING

Table 4.1 and Figure 4.1 give the overall accuracy of every model over the 100 fold-runs, next to the values published in the paper.

Table 4.1: Overall accuracy (%): Reference Paper Result vs Our Reproduction Result

Model (paper's learner)	Reference Paper Result	Our Reproduction Result	Our rank
Random Forest	80.71	71.03	1
Weighted ensemble, hard vote	–	69.93	2
Weighted ensemble, soft vote	**89.73**	**69.01**	3
Gradient Boosting (RNN)	61.43	67.67	4
SVM	67.30	61.95	5
MLP (ELM)	55.77	61.35	6

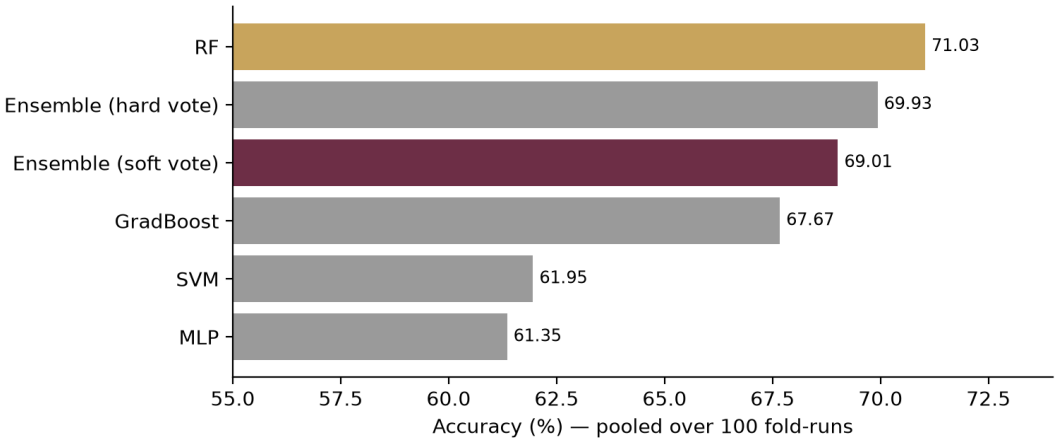


Figure 4.1: Overall accuracy of the six models (the soft-vote ensemble dark, Random Forest gold)

The central claim of the paper does not reproduce. The paper's ensemble beats its best learner by 9.02 points; ours reaches 69.01 % and stays 2.02 points *below* Random Forest. It does beat the other three learners, which is what an average of one strong and three weaker members does. The hard vote (69.93 %) is slightly higher than the soft vote overall; the soft vote is kept as the headline because it is the literal reading of the paper's equation, not because it scores higher. Section 4.4 explains the gap.

4.2 PER-CLASS COMPARISON WITH THE BASE PAPER

Table 4.2 compares the per-class results under the paper's definition (recall, Section 2.6). The shape of the paper's finding reproduces: grades 5 and 6 are predicted far better than in Cortez et al., and the extreme grades remain the hardest. The rare grades, however, are almost never recognised: grade 3 at

0 %, grade 4 at 0.57 % and grade 8 at 11.11 %. With ten grade-3 wines spread over ten folds, each test fold holds about one of them.

Table 4.2: Per-class recall (%) – the paper's "precision": benchmark, Reference Paper Result and Our Reproduction Result

Grade	Wines	Cortez et al. (2009)	Reference Paper Result	Our ensemble	Our Random Forest
3	10	0.00	33.33	0.00	0.00
4	53	20.00	63.16	0.57	0.00
5	681	67.50	84.80	81.66	81.97
6	638	57.70	88.06	69.87	74.29
7	199	58.60	77.08	49.90	51.06
8	18	0.00	50.00	11.11	11.11
Overall	**1,599**	–	**89.73**	**69.01**	**71.03**

Table 4.3 gives the conventional precision, recall and specificity of the ensemble on the pooled predictions, and Figure 4.2 the pooled confusion matrices. Most errors fall into the neighbouring grade, and the rare grades are absorbed by grades 5 and 6.

Table 4.3: Our Reproduction Result: conventional per-class metrics of the soft-vote ensemble (%, pooled predictions)

Grade	Predictions (10 × wines)	Precision	Recall	Specificity
3	100	0.00	0.00	99.95
4	530	7.50	0.57	99.76
5	6,810	72.69	81.66	77.24
6	6,380	65.61	69.87	75.68
7	1,990	68.11	49.90	96.68
8	180	51.28	11.11	99.88

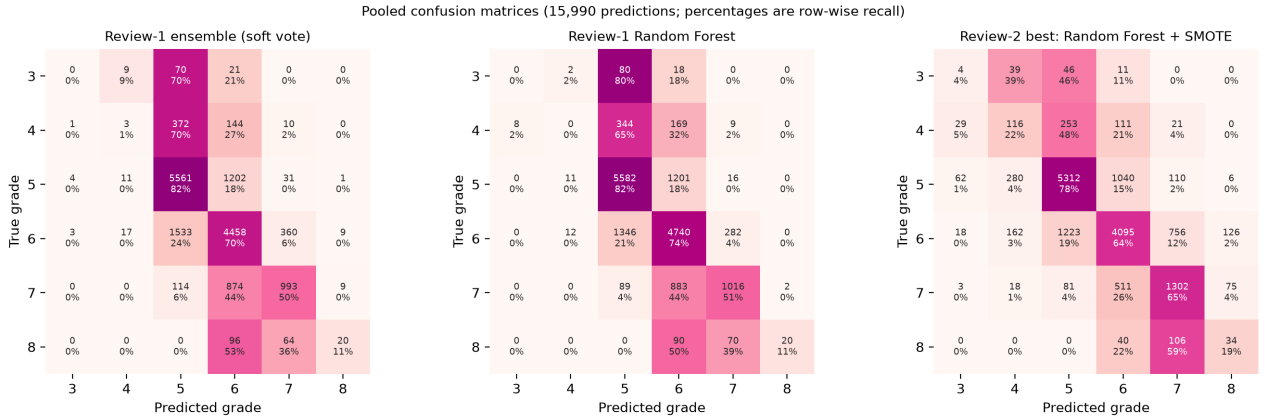


Figure 4.2: Pooled confusion matrices of the Review-1 ensemble, the Review-1 Random Forest and the best Review-2 model

4.3 LEARNED WEIGHTS AND THE PAPER'S WEIGHTS

Table 4.4 and Figure 4.3 compare the final weights (3.14). Ours never differ from the uniform 0.25 by more than 0.020; the paper's never by more than 0.053. In both cases the scheme of Section 3.6 produces nearly equal weights.

Table 4.4: Final ensemble weights: Reference Paper Result (Table 8 of the paper) vs Our Reproduction Result (mean of 10 repeats)

Learner (paper's learner)	Reference Paper Result	Our Reproduction Result
SVM	0.2516	0.2367
Random Forest	0.3030	0.2697
Gradient Boosting (RNN)	0.2286	0.2570
MLP (ELM)	0.2168	0.2367

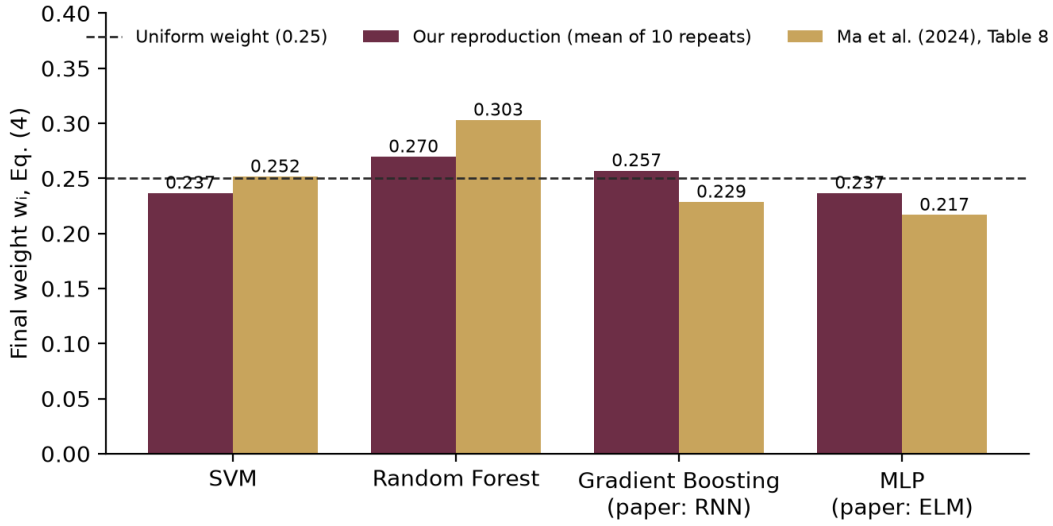


Figure 4.3: Final weights of the four learners: our reproduction and the paper's Table 8

4.4 WHY 89.73 % DOES NOT REPRODUCE

Two checks, computed in code from the numbers printed in the paper, show that the published figures cannot be reconciled with each other. They are reported as figures we *could not reconcile*, not as a claim that the paper is wrong; the paper's confusion matrix was transcribed by eye from the printed table and may contain reading errors.

(a) The confusion matrix implies a different protocol and accuracy. The cells of the paper's ensemble confusion matrix (its Table 9) add up to 491 wines – 30.71 % of the data, that is a single hold-out split rather than 100 cross-validation runs over all wines. Its diagonal holds 403 correct predictions, an accuracy of 82.08 % – 7.65 points below the 89.73 % stated in the text (Table 4.5).

Table 4.5: Check (a): what the paper's own confusion matrix implies

Quantity	Value
Wines in the confusion matrix	491
Share of the 1,599 wines	30.71 %
Correct predictions (diagonal)	403
Accuracy implied by the matrix	82.08 %
Accuracy reported in the text	89.73 %
Difference	7.65 points
Per-class values reproduced column-wise (recall)	4 of 6

Table 4.6: Per-class values of the paper recomputed from its confusion matrix (%)

Grade	Reported in the paper	Divided by true totals (recall)	Divided by predicted totals (precision)	Matches
3	33.33	33.33	33.33	yes
4	63.16	63.16	54.55	yes
5	84.80	84.80	86.93	yes
6	88.06	88.50	87.62	yes
7	77.08	61.67	60.66	no
8	50.00	60.00	75.00	no

(b) The paper's weights and base accuracies imply about 67.5 %. Applying the paper's final weights to its own base accuracies gives a weighted average of 67.52 % (Table 4.7). A voting ensemble can legitimately exceed this average when its members err on different wines, so this is an indicative reference point, not a proof. But the paper's ensemble sits 22.21 points above it, while ours sits 1.49 points above it.

Table 4.7: Check (b): the paper's weights applied to its own base accuracies

Quantity	Value
Largest deviation of the paper's weights from 0.25	0.053
Best single learner in the paper (RF)	80.71 %
Weighted average of the paper's base accuracies	67.52 %
Paper's reported ensemble accuracy	89.73 %
Gap: paper's ensemble – weighted average	+22.21 points
Our ensemble	69.01 %
Gap: our ensemble – the paper's weighted average	+1.49 points

These checks also explain our own ranking. With weights this close to uniform, the ensemble approximates an unweighted average of its members, and an average of one strong learner and three

weaker ones lands between them – above the three weaker learners and below the strongest, exactly as in Table 4.1. Published accuracies for the six-grade red-wine task are typically in the low 60s to low 70s per cent; our 69.01 % lies in that range and 89.73 % well above it.

4.5 REVIEW 2: CLASS BALANCING

Table 4.8 applies the four balancing schemes to the paper's ensemble. Balancing moves recall from the two majority grades to the rare ones. With SMOTE, grade-4 recall rises from 0.57 % to 26.42 %, grade 3 from 0 % to 10.00 % and grade 7 from 49.90 % to 64.47 %, and macro-F1 from 36.92 to 39.34; accuracy falls by 5.15 points, because more wines of grades 5 and 6 are now assigned to the rare grades. This is the trade-off balancing is meant to make, and it is the reason accuracy alone cannot judge it.

Table 4.8: Class balancing applied to the paper's ensemble (%)

Scheme	Accuracy	Bal. acc.	Macro-F1	R_3	R_4	R_5	R_6	R_7	R_8
No balancing	68.82	35.28	36.64	0.0	0.2	81.5	69.9	49.0	11.1
Review 1 (SVM + RF only)	69.01	35.52	36.92	0.0	0.6	81.7	69.9	49.9	11.1
Class weights, all four	68.43	37.94	38.77	0.0	7.7	80.3	65.5	61.9	12.2
SMOTE oversampling	63.86	41.44	39.34	10.0	26.4	73.0	59.2	64.5	15.6

Table 4.9 shows the macro-F1 of every learner under every scheme. SMOTE helps every learner, most of all SVM and Gradient Boosting; class weights help the ensemble and Gradient Boosting but hurt the MLP, whose training becomes unstable when ten wines of grade 3 carry the weight of several hundred. Figure 4.4 shows the per-class recalls.

Table 4.9: Macro-F1 of every learner under every balancing scheme

Model	No balancing	Review 1 (partial)	Class weights, all four	SMOTE
SVM	29.30	28.01	28.01	34.24
Random Forest	38.18	38.09	38.09	40.92
Gradient Boosting	36.72	36.72	37.78	38.08
MLP	30.07	30.07	29.00	34.21
Paper's ensemble	36.64	36.92	38.77	39.34

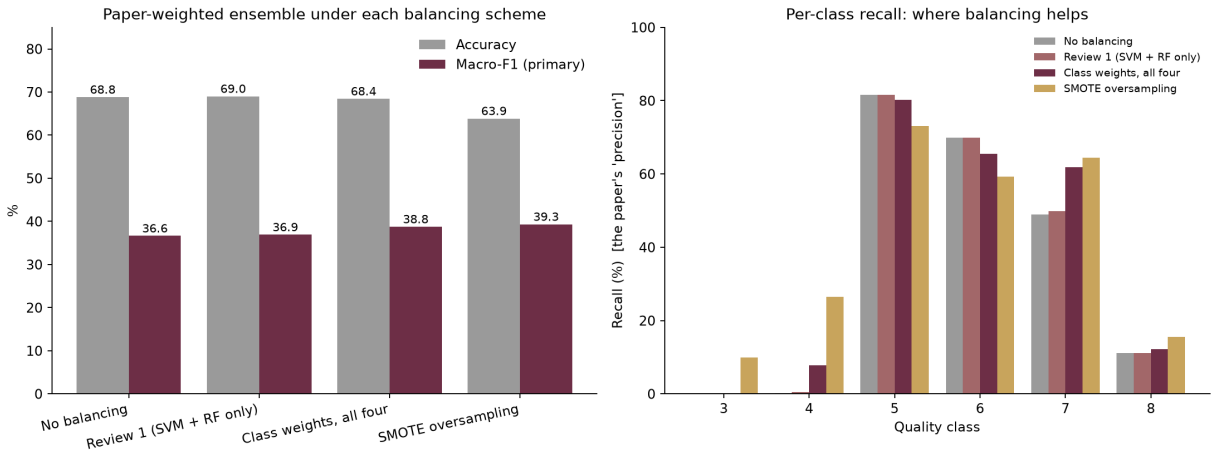


Figure 4.4: Effect of the four balancing schemes on accuracy, macro-F1 and per-class recall

4.6 REVIEW 2: IMPROVING THE RESULT

4.6.1 Testing the Review-1 Diagnosis

Review 1 attributed the ensemble's failure to beat Random Forest to its near-uniform weights. Table 4.10 and Figure 4.5 test this with the temperature of Section 3.8. As T grows, the weight moves to Random Forest and the ensemble accuracy rises; sharpening the accuracy term overtakes Random Forest (71.03 %) from T = 25. Sharpening the diversity term as well eventually backfires: it moves weight to the learner whose errors differ most from the others, which is usually the weakest. The diagnosis therefore holds – but this sweep is scored on the same predictions it is computed from, so it is only a test, not a model.

Table 4.10: Weight sharpening of the Review-1 ensemble (diagnostic only)

T	Accuracy, accuracy term sharpened (%)	Largest weight	Accuracy, both terms sharpened (%)	Macro-F1, both terms
1	69.01	0.270	69.01	36.92
2	69.22	0.293	69.20	37.03
5	69.90	0.364	69.82	37.36
10	70.78	0.466	70.59	37.93
25	71.22	0.590	71.08	38.27
100	71.23	0.623	70.41	37.85

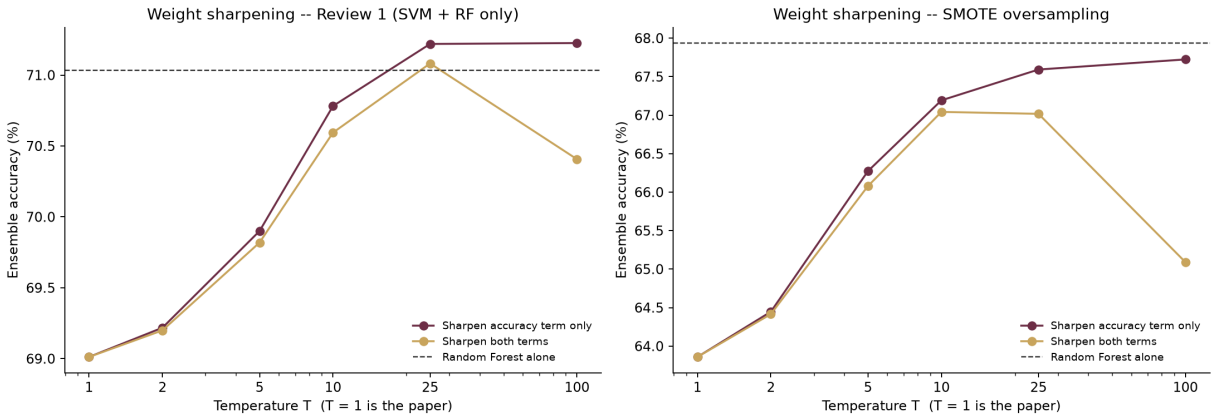


Figure 4.5: Ensemble accuracy against the temperature T (T = 1 is the paper)

4.6.2 Learned Temperature

Choosing T inside each training fold (Table 4.11) turns the diagnosis into an honest improvement. With the reproduction's own members, accuracy rises from 69.01 % to 70.89 % and macro-F1 from 36.92 to 38.08; the inner cross-validation chose a sharp temperature ($T \geq 10$) in 99 % of the 100 folds (Figure 4.6). With SMOTE members it reaches a macro-F1 of 40.87.

Table 4.11: Learned-temperature ensemble against the paper's weights (T = 1)

Members	Learned T: accuracy	Learned T: macro-F1	Paper: accuracy	Paper: macro-F1	Folds with T ≥ 10
Review-1 members	70.89	38.08	69.01	36.92	99 %
SMOTE members	66.63	40.87	63.86	39.34	94 %

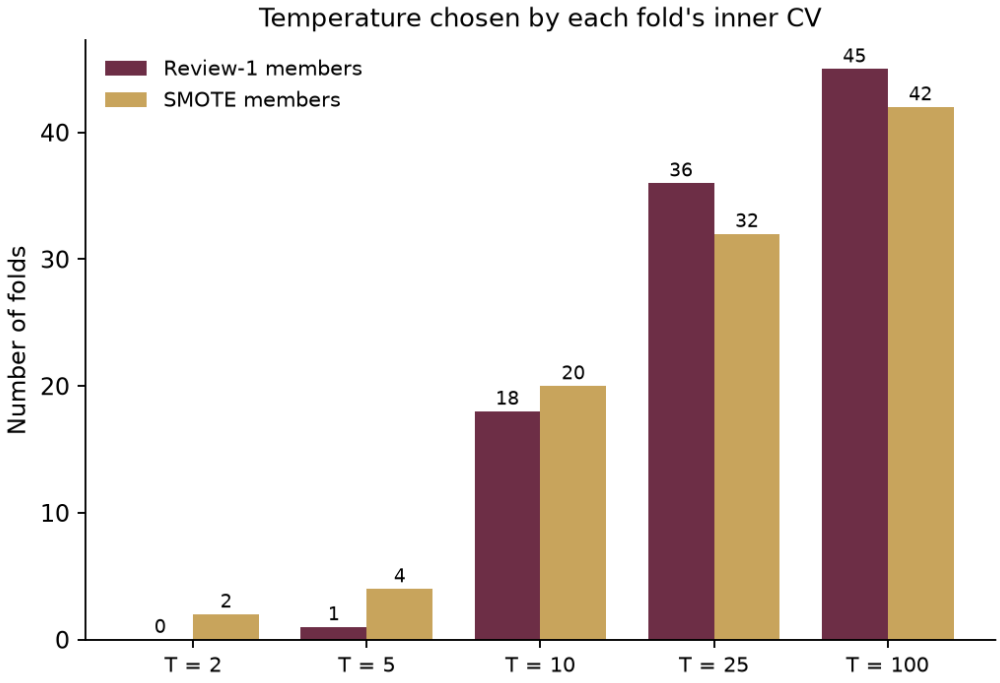


Figure 4.6: Temperature chosen by the inner cross-validation of each training fold

4.6.3 Stacking and Stronger Members

A logistic-regression combiner on the four learners reaches 70.92 % accuracy but only 34.96 macro-F1: trained to maximise likelihood on imbalanced data, it relearns the majority-grade bias. Its class-balanced version swings the other way (59.20 % accuracy, the highest balanced accuracy of all at 50.03 %, with 48.00 % recall on grade 3 and 53.33 % on grade 8). ExtraTrees alone (70.97 %) is as accurate as Random Forest, and the paper's weights on SVM + RF + ExtraTrees + HistGB give 70.79 %.

4.6.4 All Candidates

Table 4.12 ranks all 28 candidates by the primary metric, and Figure 4.7 plots accuracy against macro-F1. No candidate is best on both. The best macro-F1 comes from **Random Forest with SMOTE** (40.92), followed closely by the learned-temperature ensemble with SMOTE (40.87); the best accuracy is still Random Forest without resampling (71.03 %). The dashed line in Figure 4.7 joins the models that no other model beats on both metrics.

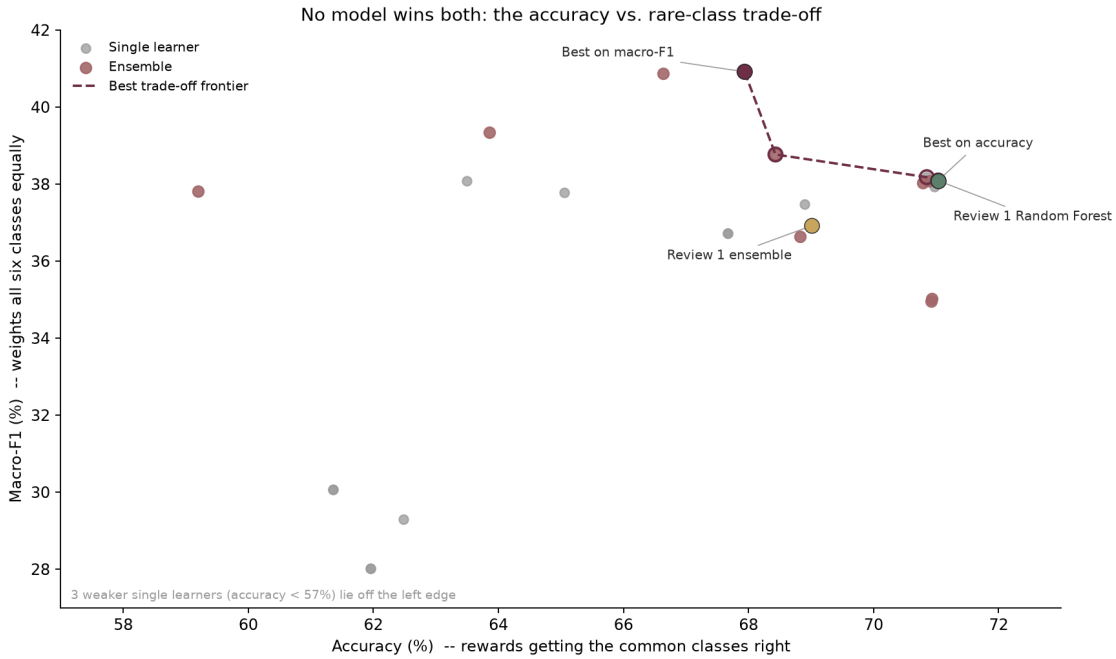


Figure 4.7: Accuracy against macro-F1 for every candidate; the dashed line is the best trade-off frontier

Table 4.12: All Review-2 candidates ranked by macro-F1 (%, 100 fold-runs each)

Rank	Model	Kind	Accuracy	Bal. acc.	Macro-F1
1	**Random Forest · SMOTE**	**single learner**	**67.94**	**42.07**	**40.92**
2	Learned-T ensemble · SMOTE	ensemble	66.63	42.49	40.87
3	Paper ensemble · SMOTE	ensemble	63.86	41.44	39.34
4	Paper ensemble · class weights (all four)	ensemble	68.43	37.94	38.77
5	Random Forest · no balancing	single learner	70.85	36.70	38.18
6	Random Forest · Review-1 setup	single learner	71.03	36.40	38.09
7	Random Forest · class weights (all four)	single learner	71.03	36.40	38.09
8	Gradient Boosting · SMOTE	single learner	63.49	39.40	38.08
9	Learned-T ensemble · Review-1 setup	ensemble	70.89	36.58	38.08
10	Paper weights: SVM + RF + ExtraTrees + HistGB	ensemble	70.79	36.96	38.04
11	ExtraTrees	single learner	70.97	36.45	37.93
12	Stacked: paper's 4 learners, balanced combiner	ensemble	59.20	50.03	37.82
13	Gradient Boosting · class weights (all four)	single learner	65.05	37.92	37.78
14	HistGB	single learner	68.89	36.86	37.48
15	Paper ensemble · Review-1 setup	ensemble	69.01	35.52	36.92
16	Gradient Boosting · no balancing	single learner	67.67	35.70	36.72
17	Gradient Boosting · Review-1 setup	single learner	67.67	35.70	36.72
18	Paper ensemble · no balancing	ensemble	68.82	35.28	36.64
19	Stacked: SVM + RF + ExtraTrees + HistGB	ensemble	70.93	34.92	35.02
20	Stacked: paper's 4 learners (class-weighted)	ensemble	70.92	35.04	34.96
21	SVM · SMOTE	single learner	54.80	38.95	34.24
22	MLP · SMOTE	single learner	54.82	37.26	34.21
23	MLP · no balancing	single learner	61.35	29.75	30.07
24	MLP · Review-1 setup	single learner	61.35	29.75	30.07
25	SVM · no balancing	single learner	62.48	29.28	29.30
26	MLP · class weights (all four)	single learner	44.57	39.55	29.00
27	SVM · Review-1 setup	single learner	61.95	27.99	28.01
28	SVM · class weights (all four)	single learner	61.95	27.99	28.01

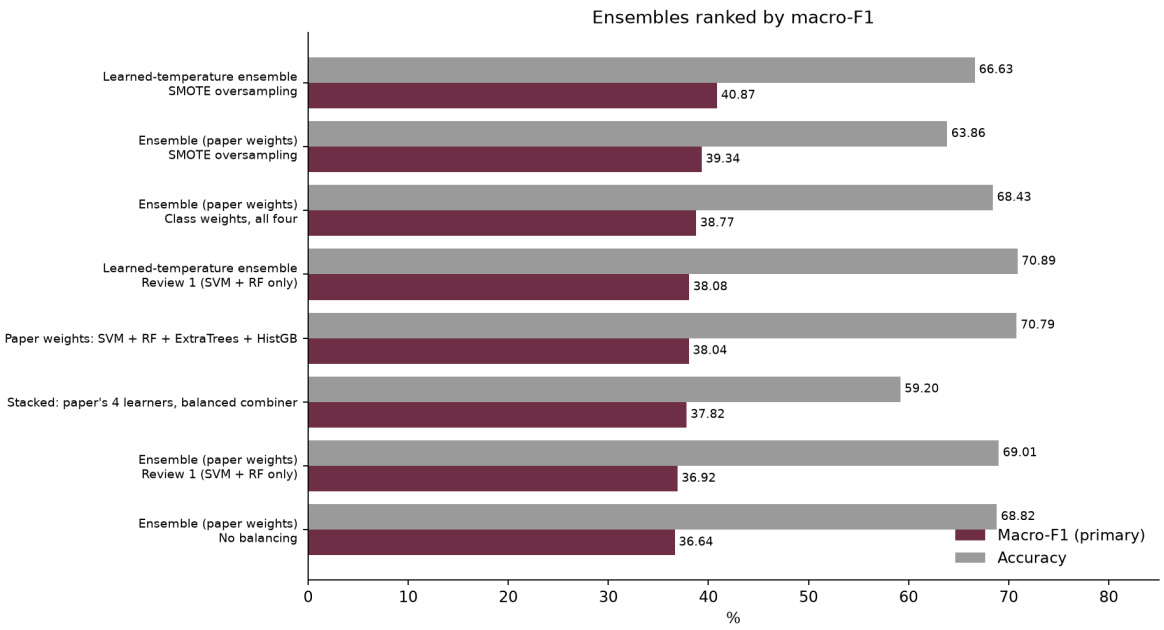


Figure 4.8: The leading candidates on accuracy, balanced accuracy and macro-F1

4.7 CONSISTENCY ACROSS REPEATS

Every candidate was scored on identical folds in each repeat, so their per-repeat scores can be paired (Table 4.13). Random Forest with SMOTE beats the Review-1 ensemble on macro-F1 in 10 of 10 repeats and loses on accuracy in 9 of 10; the learned-temperature ensemble beats the paper's ensemble on both metrics in all 10 repeats. The repeats reuse the same 1,599 wines with different splits, so they are not independent samples and the Wilcoxon signed-rank p-values [31] are optimistic [23], [8]; the win counts are the plainer evidence.

Table 4.13: Paired comparison over the 10 repeats

Model	Compared with	Metric	Mean diff.	Repeats won	Wilcoxon p
Random Forest · SMOTE	Paper ensemble · Review-1 setup	macro-F1	+3.97	10 / 10	0.002
Random Forest · SMOTE	Paper ensemble · Review-1 setup	accuracy	-1.08	1 / 10	0.004
Random Forest · SMOTE	Random Forest · Review-1 setup	macro-F1	+2.80	10 / 10	0.002
Random Forest · SMOTE	Random Forest · Review-1 setup	accuracy	-3.10	0 / 10	0.002
Random Forest · SMOTE	Learned-T ensemble · SMOTE	macro-F1	+0.01	4 / 10	0.846
Random Forest · SMOTE	Learned-T ensemble · SMOTE	accuracy	+1.31	9 / 10	0.006
Learned-T ensemble · Review-1 setup	Paper ensemble · Review-1 setup	macro-F1	+1.16	10 / 10	0.002
Learned-T ensemble · Review-1 setup	Paper ensemble · Review-1 setup	accuracy	+1.88	10 / 10	0.002
Learned-T ensemble · Review-1 setup	Random Forest · Review-1 setup	macro-F1	-0.01	6 / 10	0.922
Learned-T ensemble · Review-1 setup	Random Forest · Review-1 setup	accuracy	-0.14	4 / 10	0.438
Random Forest · Review-1 setup	Paper ensemble · Review-1 setup	macro-F1	+1.17	10 / 10	0.002
Random Forest · Review-1 setup	Paper ensemble · Review-1 setup	accuracy	+2.02	10 / 10	0.002

4.8 FEATURE IMPORTANCE

Table 4.14 gives the three importance measures of Section 3.11 for the best model (Random Forest with SMOTE) and for the Review-1 Random Forest. Permutation importance is the macro-F1 lost, in points, when a feature is shuffled on the held-out folds. The three rankings agree closely (pairwise rank correlation 0.74–0.89): **alcohol, sulphates and volatile acidity** are clearly the most important, followed by chlorides and total sulphur dioxide. Three of the four attributes the paper highlights are in the top three; citric acid ranks sixth. Higher alcohol, sulphates and citric acid go with better wine, higher volatile acidity with worse (Figures 4.9 and 4.10). Small negative values mean that shuffling a feature made no difference beyond noise; these features are partly redundant with others, such as density with alcohol.

Table 4.14: Feature importance: permutation importance (macro-F1 points), MDI and Spearman correlation

Rank	Feature	Perm., best model	Perm., RF	MDI, RF	Spearman ρ	Paper's key attribute
1	alcohol	7.45	8.48	0.135	+0.48	yes
2	sulphates	7.28	6.49	0.119	+0.38	yes
3	volatile acidity	4.44	4.86	0.129	-0.38	yes
4	chlorides	2.33	1.05	0.093	-0.19	
5	total sulfur dioxide	1.66	2.27	0.089	-0.20	
6	citric acid	0.56	1.39	0.079	+0.21	yes
7	fixed acidity	-0.13	0.34	0.068	+0.11	
8	residual sugar	-0.15	0.34	0.065	+0.03	
9	density	-0.50	0.67	0.085	-0.18	
10	pH	-0.59	0.92	0.076	-0.04	
11	free sulfur dioxide	-1.16	1.06	0.062	-0.06	

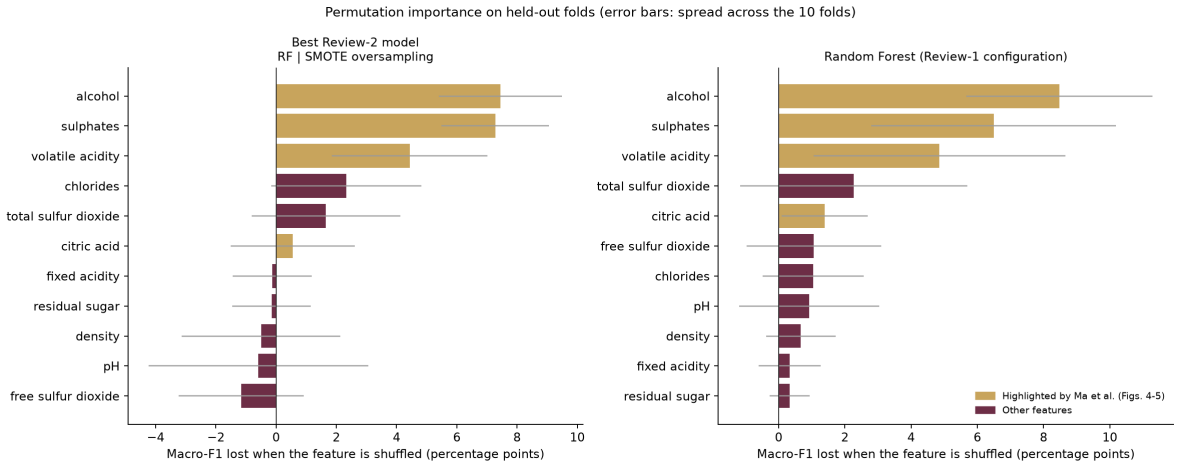


Figure 4.9: Permutation importance on the held-out folds (macro-F1 lost when a feature is shuffled)

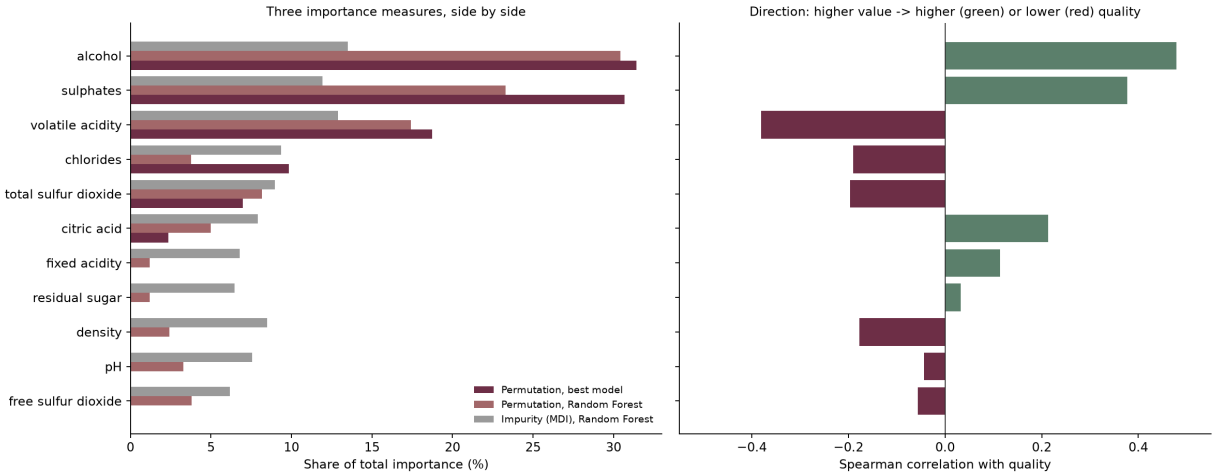


Figure 4.10: The three importance measures side by side, with the direction of each effect

How many laboratory tests are needed? Table 4.15 and Figure 4.11 retrain Random Forest on the top-k features chosen inside each training fold. Four tests – alcohol, volatile acidity, sulphates and chlorides – already give 69.27 % accuracy, within 1.76 points of all eleven, and the best macro-F1 of the curve. A retailer could therefore screen wines with a few routine tests.

Table 4.15: Random Forest on the top-k features chosen inside each training fold (%)

Subset	Tests	Accuracy	Macro-F1	Features chosen most often
top 1	1	28.98	18.51	alcohol
top 2	2	58.93	32.65	alcohol; volatile acidity
top 3	3	66.25	36.77	volatile acidity; sulphates; alcohol
top 4	4	69.27	38.46	volatile acidity; sulphates; alcohol; chlorides
top 6	6	70.21	38.35	volatile acidity; sulphates; alcohol; total sulfur dioxide; chlorides; density
top 8	8	70.79	38.23	volatile acidity; citric acid; chlorides; total sulfur dioxide; density; sulphates; alcohol; pH
top 11	11	71.03	38.09	fixed acidity; volatile acidity; citric acid; residual sugar; chlorides; free sulfur dioxide; total sulfur dioxide; density; pH; sulphates; alcohol
Ma et al.'s 4 key attributes	4	67.55	37.68	alcohol; sulphates; volatile acidity; citric acid

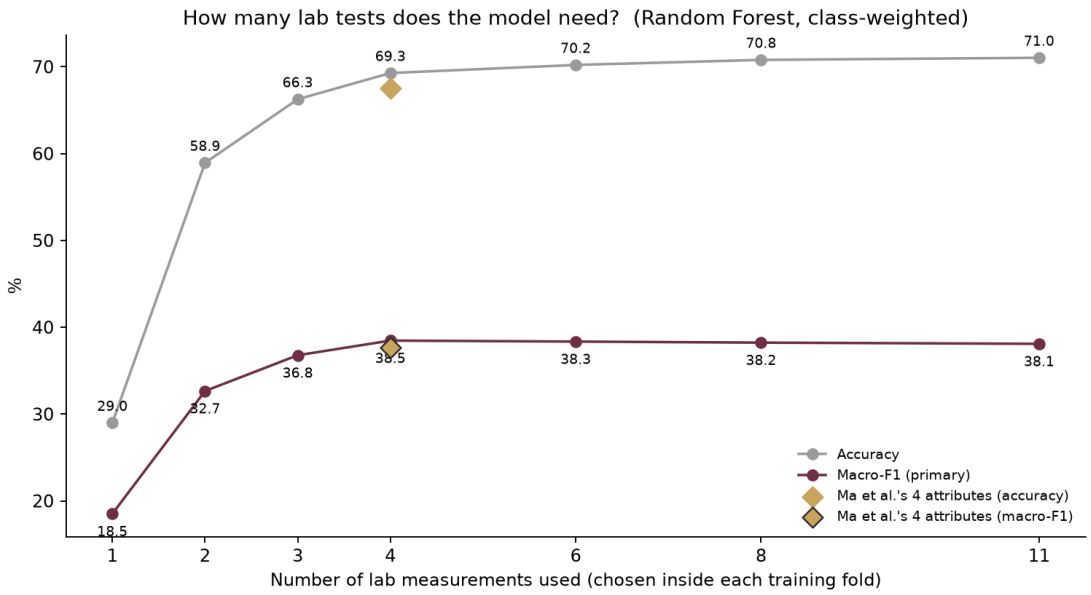


Figure 4.11: Accuracy and macro-F1 against the number of laboratory tests used

4.9 REFERENCE PAPER RESULT VS OUR RESULT

Table 4.16 brings the results together. The balanced accuracy of the paper is computed here as the mean of its per-class values; the paper does not report macro-F1.

Table 4.16: Reference Paper Result vs Our Reproduction Result (Review 1) vs the best Review-2 model

Metric	Reference Paper Result	Review-1 ensemble	Review-1 Random Forest	Review-2 best (RF + SMOTE)
Accuracy (%)	89.73	69.01	71.03	67.94
Balanced accuracy (%)	66.07	35.52	36.40	42.07
Macro-F1 (%)	not reported	36.92	38.09	40.92
Recall, grade 3 (%)	33.33	0.00	0.00	4.00
Recall, grade 4 (%)	63.16	0.57	0.00	21.89
Recall, grade 5 (%)	84.80	81.66	81.97	78.00
Recall, grade 6 (%)	88.06	69.87	74.29	64.19
Recall, grade 7 (%)	77.08	49.90	51.06	65.43
Recall, grade 8 (%)	50.00	11.11	11.11	18.89

4.10 DISCUSSION

Three findings stand out. First, the weighted ensemble of the base paper was implemented as specified, with leakage-free evaluation, and reaches 69.01 % – inside the range this dataset normally gives and within 1.49 points of what the paper's own weights and base accuracies imply. The 89.73 % could not be reconciled with the paper's own tables. The cause on our side is structural, not an implementation detail: normalising inverse errors and error distances of similar learners gives nearly equal weights, so the ensemble averages its members instead of exceeding the best of them –

consistent with the observation of Kuncheva and Whitaker [19] that diversity does not translate simply into accuracy.

Second, class balancing does what it should. Accuracy rewards predicting grades 5 and 6; balancing gives up some of that to recognise the rare grades, and the best model raises macro-F1 by 4.00 points and grade-4 recall by 21.32 points over the Review-1 ensemble. Grade 3 remains almost unpredictable with ten examples. Which model to deploy depends on the use: Random Forest for the most correct grades overall, Random Forest with SMOTE to find the unusual wines – both are offered in the web application.

Third, the Review-1 diagnosis held when tested: sharpening the weights lets the ensemble overtake Random Forest, and choosing the sharpness inside the training folds improves the paper's ensemble in every repeat. Three independent importance measures agree on what drives the prediction, and they confirm three of the paper's four key attributes. None of the configurations approaches 89.73 %: the best accuracy of any honest configuration is 71.03 %, which strengthens the Review-1 conclusion.

CHAPTER 5
WEB APPLICATION: DESIGN AND DEPLOYMENT

5.1 PURPOSE OF THE WEB APPLICATION

The analysis lives in Python modules and result files, which is convenient for development but not for a reviewer or a wine buyer. The project therefore includes an interactive web application built with Streamlit [27] and Altair charts [29]. It presents every result of the project and predicts the quality of new wines without any programming. Its source code is available at https://github.com/Surianandhan/wine-quality-ml-project, from which it runs locally or on Streamlit Community Cloud (Section 5.7). Its goals are to:

- **explore** the dataset, the reproduction, the class-balancing experiment, the improvements and the feature importance with charts and tables;
- **predict** the quality grade of a wine from its 11 measurements with two models: the most accurate (Random Forest) and the best on the rare grades (Random Forest with SMOTE);
- **score in bulk:** upload a CSV file of wines and download their predicted grades; and
- **stay faithful to the project:** use exactly the project's models and numbers, which automated checks confirm.

5.2 SYSTEM ARCHITECTURE

Figure 5.1 shows how the application is built. The result pages read the tables and figures written by run_all.py, so they always show the numbers of this report. For prediction, the application imports the project's own model definitions from src/ and trains the two models once at start-up on all 1,599 wines; Streamlit's resource cache keeps them in memory, so later visits are instant. The application is deployed from the GitHub repository to Streamlit Community Cloud, and app/test_app.py checks it automatically.

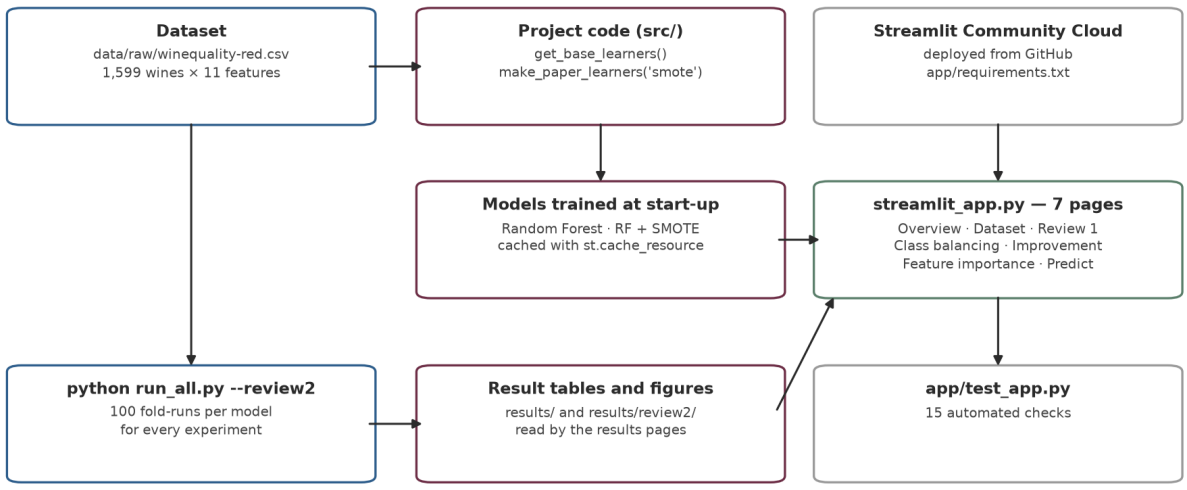


Figure 5.1: Architecture of the web application

5.3 TECHNOLOGY STACK

Table 5.1 lists the software. The library versions are pinned to exactly those with which the results were produced, so the application reproduces the project's models.

Table 5.1: Technology stack of the project

Layer	Technology	Version	Role in the project
Language	Python	3.11	models, experiments, figures and the web application
Data handling	pandas [22], NumPy [14]	3.0.5, 2.4.6	loading the data, result tables
Machine learning	scikit-learn [24], SciPy [30]	1.9.0, 1.17.1	pipelines, learners, cross-validation, metrics, statistics
Class balancing	imbalanced-learn [20]	0.14.2	SMOTE inside the training folds
Figures	Matplotlib [17]	3.11.1	figures of the experiments and of this report
Web framework	Streamlit [27]	1.66.0	pages, navigation, caching, widgets, file upload and download
Interactive charts	Altair [29]	6.3.0	bar charts of the predicted probabilities
Version control	Git and GitHub	–	public repository of the project
Hosting	Streamlit Community Cloud	–	free hosting; redeploys on every push

5.4 PAGES AND FEATURES

The application has seven pages, listed in the sidebar (Table 5.2).

Table 5.2: Pages of the web application

Page	What it shows	What the user can do
Overview	the project in brief: dataset, headline numbers of Review 1 and Review 2, a guide to the pages	navigate to every page
Dataset	the class distribution, summary statistics and the correlation of each measurement with quality	inspect the data
Review 1: reproduction	model ranking, per-class comparison with the published benchmarks and the two checks of Section 4.4	compare the reproduction with the paper
Review 2: class balancing	accuracy, macro-F1 and per-class recall under the four balancing schemes	see the effect of balancing
Review 2: improving the result	the temperature test, the learned temperature, the paired comparison and all candidates	compare every candidate model
Feature importance	the three importance measures, their agreement and the top-k laboratory-test curve	see what drives the prediction
Predict a wine	the grade predicted by both models with the probability of every grade; batch prediction	set 11 sliders or start from a typical wine; upload a CSV file

5.5 WORKED EXAMPLES

The prediction page can start from the median measurements of all wines or of the wines of one grade. Table 5.3 shows what the two models predict for these typical wines. Both models recover the grade of the typical grade-5 and grade-6 wines, and Random Forest also that of the grade-7 wine. For the typical grade-4 wine both predict 5 – the boundary that the rare grades fall across. For the typical grade-8 wine Random Forest predicts 7, while the SMOTE model predicts 8 with 84 % probability – but it also predicts 8 for the typical grade-7 wine. This is the trade-off of Chapter 4 in miniature: the SMOTE model finds the rare grades more often at the price of some errors on the common ones.

Table 5.3: Example wines in the web application: three key measurements and the predicted grade (probability)

Example (median of)	Alcohol (% vol.)	Volatile acidity	Sulphates	Random Forest	RF + SMOTE
Typical wine (median of all wines)	10.20	0.52	0.62	6 (50.3 %)	6 (53.0 %)
Typical grade-4 wine	10.00	0.67	0.56	5 (64.0 %)	5 (41.3 %)
Typical grade-5 wine	9.70	0.58	0.58	5 (52.7 %)	5 (44.7 %)
Typical grade-6 wine	10.50	0.49	0.64	6 (51.3 %)	6 (47.0 %)
Typical grade-7 wine	11.50	0.37	0.74	7 (54.3 %)	8 (57.0 %)
Typical grade-8 wine	12.15	0.37	0.74	7 (53.3 %)	8 (84.0 %)

The batch tab accepts a CSV file with the 11 measurement columns, separated by commas or semicolons, reports any missing column by name and returns the input with both predictions added. A sample file can be downloaded from the page.

5.6 VERIFICATION OF THE APPLICATION

The file app/test_app.py contains 15 automated checks in four groups (Table 5.4); all 15 pass (Figure 6.1).

Table 5.4: Automated checks of the web application

Group	Checks	What is verified
1. Pages	7	every page renders without an exception
2. Models	2	the two models give exactly the predictions of freshly fitted project models (Review-1 Random Forest; Review-2 SMOTE Random Forest), which proves identical settings
3. Predictions	4	probabilities sum to 1 and every predicted grade is a real grade, for both models
4. Numbers	2	the reproduction accuracy shown is 69.01 % and the best macro-F1 model is RF + SMOTE, as in the result files

5.7 DEPLOYMENT

The project is version-controlled in a public GitHub repository (Table 5.5) and the application is ready for Streamlit Community Cloud (the steps are in DEPLOY.md), which installs the pinned app/ requirements.txt with Python 3.11 and redeploys after every push. The base paper's PDF is excluded from the repository by .gitignore because it is a copyrighted article. The application can also be run locally with the command streamlit run app/streamlit_app.py. On the free tier the application sleeps after a period without visitors and wakes up in about a minute.

Table 5.5: Structure of the GitHub repository

Path	Contents
app/	streamlit_app.py (the seven pages), test_app.py (15 checks), requirements.txt
src/	config.py, preprocessing.py, base_learners.py, ensemble.py, evaluate.py (Review 1); review2.py, feature_importance.py (Review 2)
run_all.py	runs every experiment: python run_all.py (Review 1), python run_all.py --review2 (Review 2)
data/raw/	the UCI red- and white-wine files
results/	tables/ and figures/ of Review 1; review2/tables and review2/figures of Review 2
notebooks/	a Google Colab notebook that runs the whole project
report/	this report and the conference paper (Word and PDF), methodology.md, the figures and the scripts that build them
presentation/	the three review decks with speaker notes, the study guide and the scripts that build them
README.md, HOW_TO_RUN.md, DEPLOY.md	overview, running instructions and deployment steps

5.8 SNAPSHOTS OF THE WEB APPLICATION

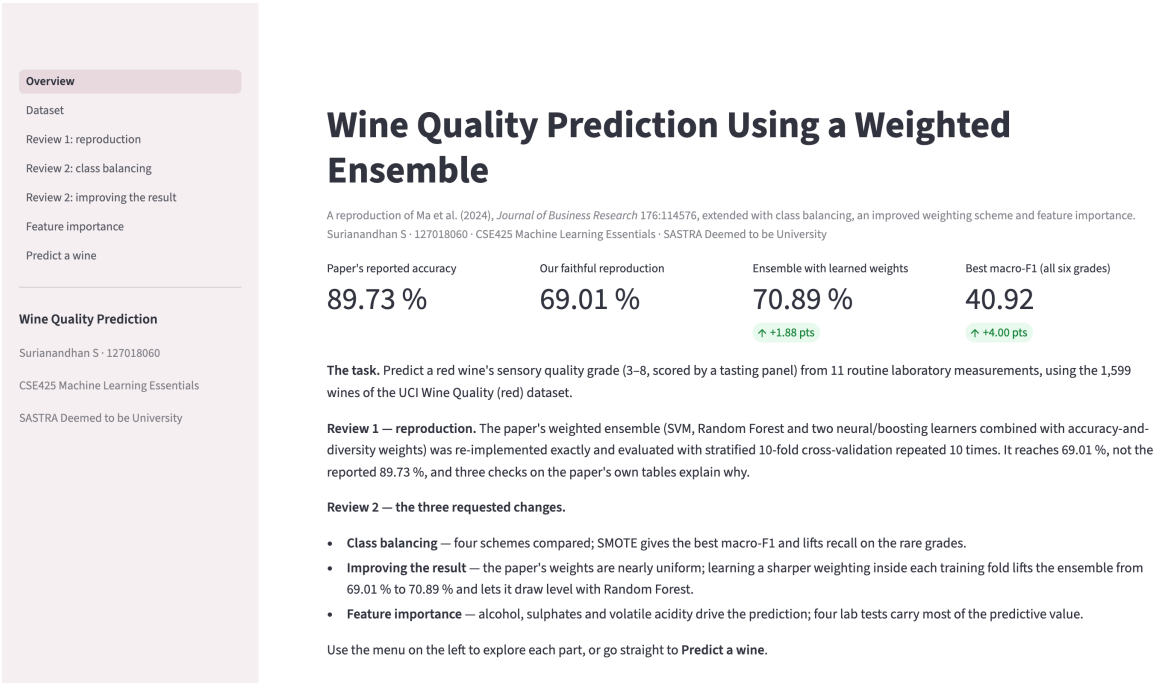


Figure 5.2: Overview page: the project in brief and the headline numbers

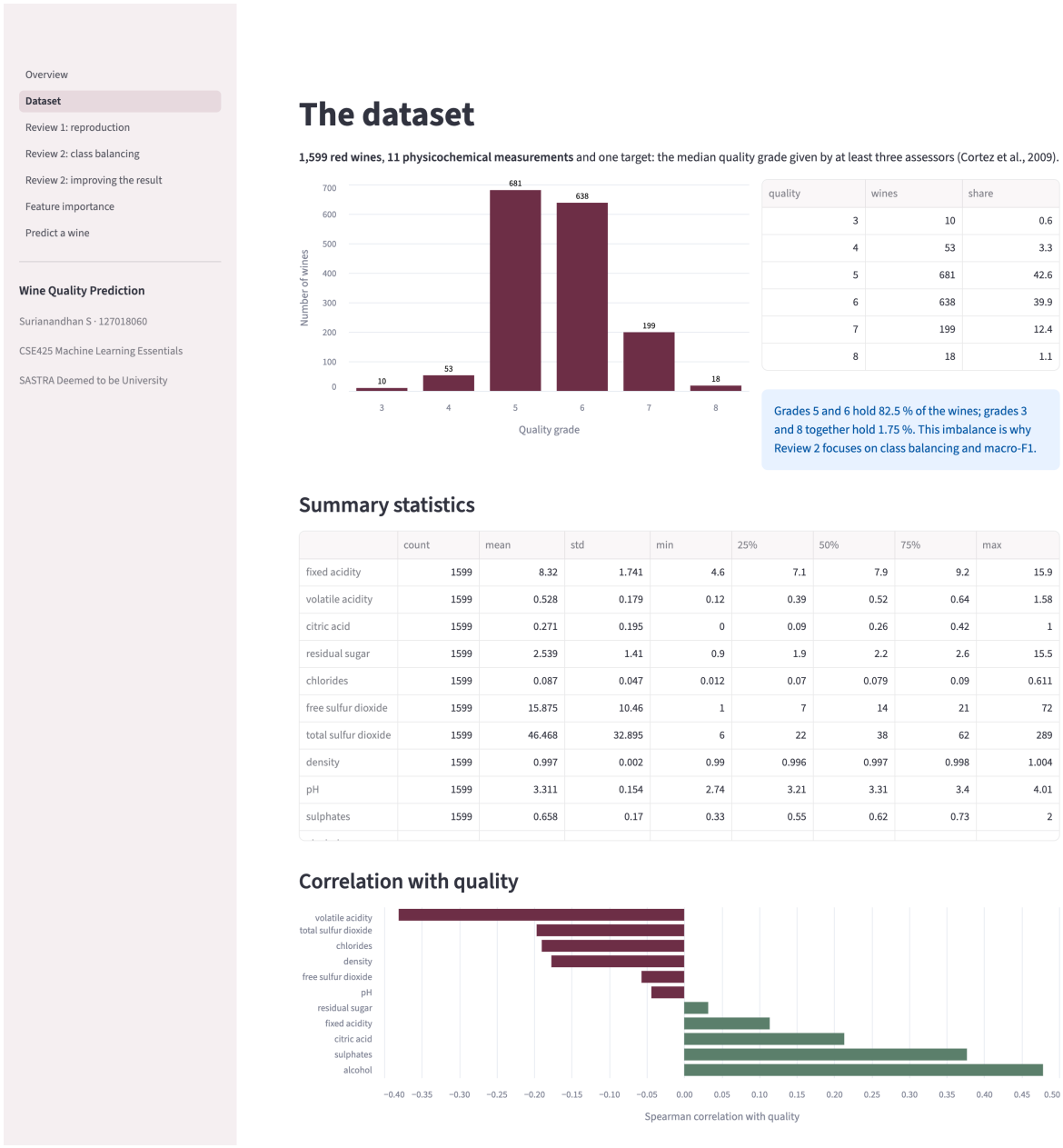


Figure 5.3: Dataset page: class distribution, summary statistics and correlations

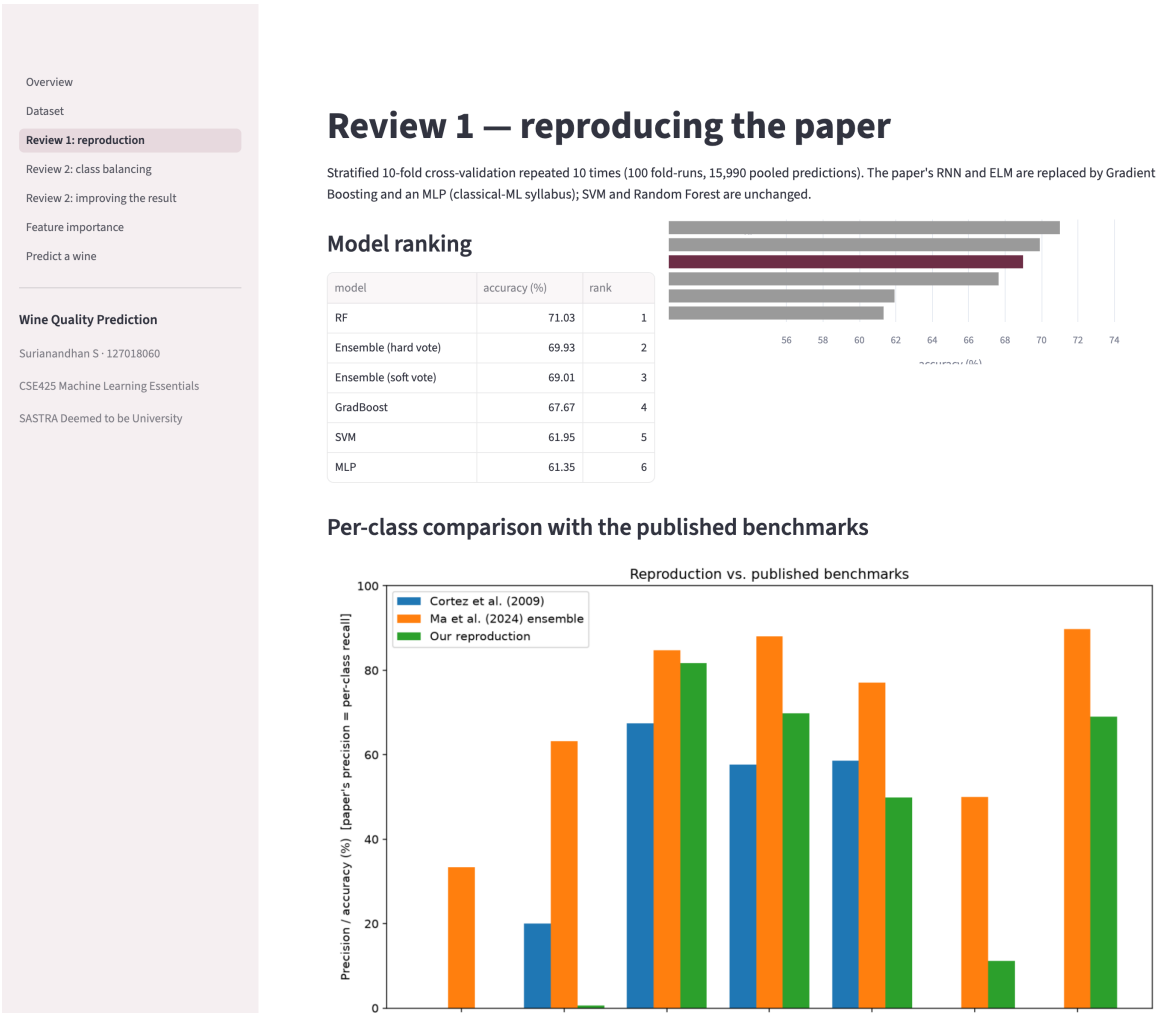
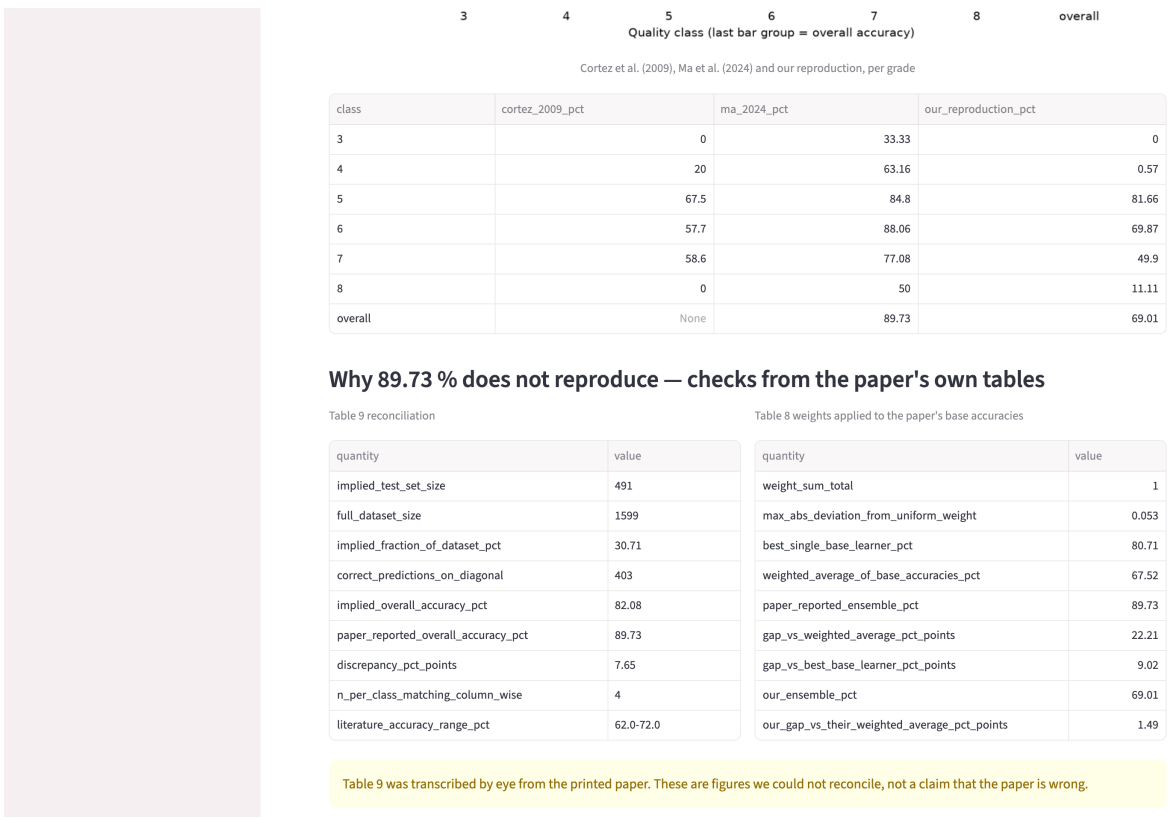


Figure 5.4: Review 1 page: model ranking and comparison with the published benchmarks (part 1 of 2)



Cortez et al. (2009), Ma et al. (2024) and our reproduction, per grade

class	cortez_2009_pct	ma_2024_pct	our_reproduction_pct
3	0	33.33	0
4	20	63.16	0.57
5	67.5	84.8	81.66
6	57.7	88.06	69.87
7	58.6	77.08	49.9
8	0	50	11.11
overall	None	89.73	69.01

Why 89.73 % does not reproduce — checks from the paper's own tables

Table 9 reconciliation

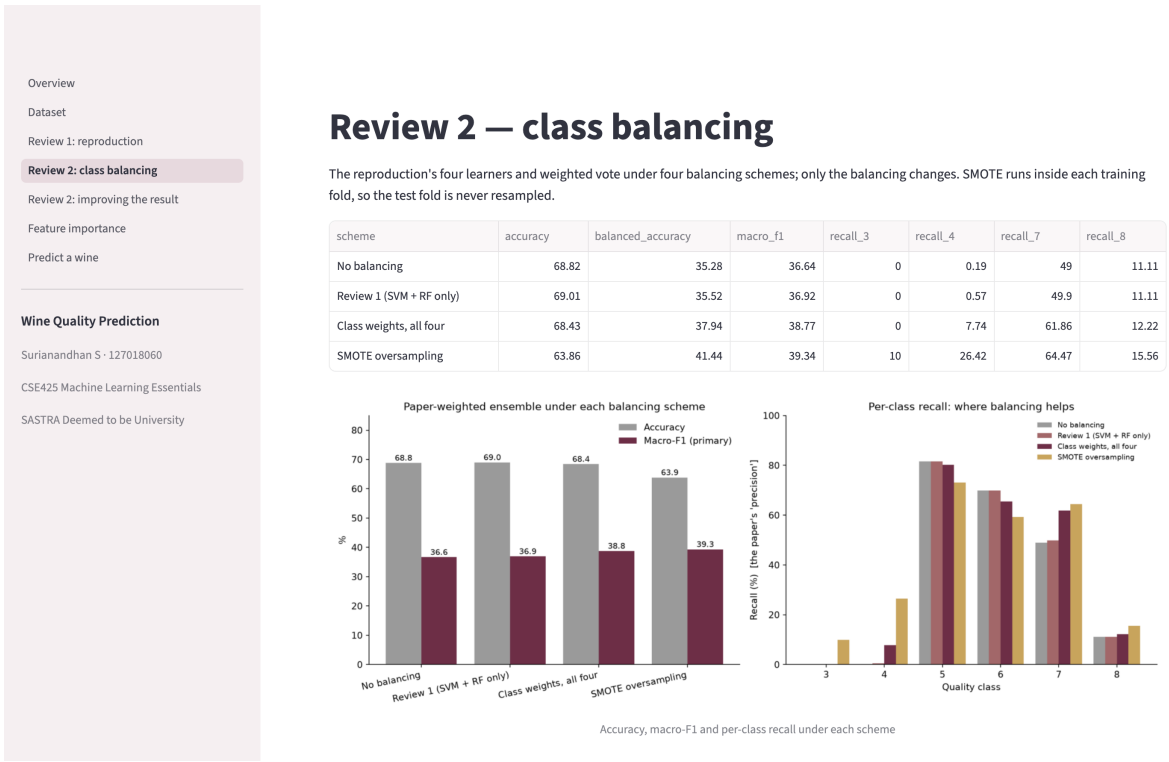
quantity	value
implied_test_set_size	491
full_dataset_size	1599
implied_fraction_of_dataset_pct	30.71
correct_predictions_on_diagonal	403
implied_overall_accuracy_pct	82.08
paper_reported_overall_accuracy_pct	89.73
discrepancy_pct_points	7.65
n_per_class_matching_column_wise	4
literature_accuracy_range_pct	62.0-72.0

Table 8 weights applied to the paper's base accuracies

quantity	value
weight_sum_total	1
max_abs_deviation_from_uniform_weight	0.053
best_single_base_learner_pct	80.71
weighted_average_of_base_accuracies_pct	67.52
paper_reported_ensemble_pct	89.73
gap_vs_weighted_average_pct_points	22.21
gap_vs_best_base_learner_pct_points	9.02
our_ensemble_pct	69.01
our_gap_vs_their_weighted_average_pct_points	1.49

Table 9 was transcribed by eye from the printed paper. These are figures we could not reconcile, not a claim that the paper is wrong.

Figure 5.5: Review 1 page: model ranking and comparison with the published benchmarks (part 2 of 2)



scheme	accuracy	balanced_accuracy	macro_f1	recall_3	recall_4	recall_7	recall_8
No balancing	68.82	35.28	36.64	0	0.19	49	11.11
Review 1 (SVM + RF only)	69.01	35.52	36.92	0	0.57	49.9	11.11
Class weights, all four	68.43	37.94	38.77	0	7.74	61.86	12.22
SMOTE oversampling	63.86	41.44	39.34	10	26.42	64.47	15.56

Accuracy, macro-F1 and per-class recall under each scheme

Figure 5.6: Review 2 page: class balancing

Overview

Dataset

Review 1: reproduction

Review 2: class balancing

Review 2: improving the result

Feature importance

Predict a wine

Wine Quality Prediction

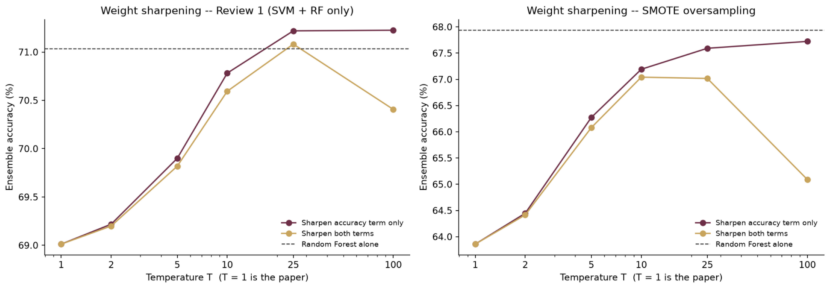
Surianandhan S · 127018060

CSE425 Machine Learning Essentials

SASTRA Deemed to be University

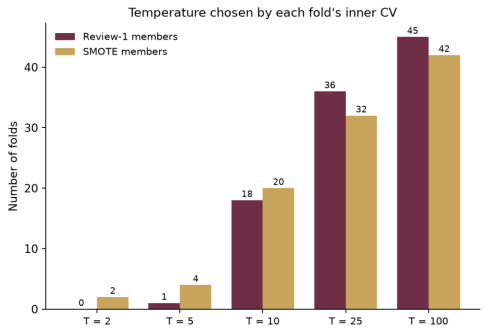
Review 2 — improving the result

1 · Testing the Review-1 diagnosis: sharpening the weights



Weight temperature T (T = 1 is the paper). Diagnostic only.

2 · The fix: temperature learned inside each training fold



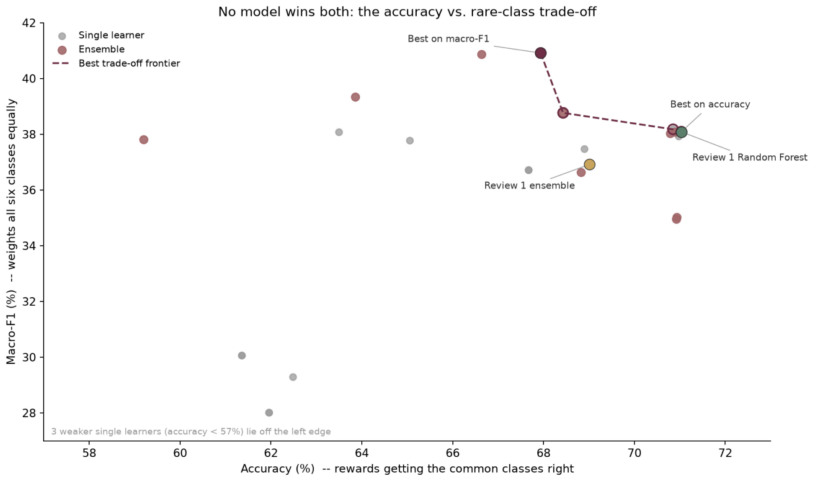
Temperature chosen by each fold's inner cross-validation

model	compared_with
RF · SMOTE	Paper ensemble · Review
RF · SMOTE	Paper ensemble · Review
RF · SMOTE	RF · Review-1 setup
RF · SMOTE	RF · Review-1 setup
RF · SMOTE	Learned-T ensemble · SM
RF · SMOTE	Learned-T ensemble · SM
Learned-T ensemble · Review-1 setup	Paper ensemble · Review
Learned-T ensemble · Review-1 setup	Paper ensemble · Review
Learned-T ensemble · Review-1 setup	RF · Review-1 setup
Learned-T ensemble · Review-1 setup	RF · Review-1 setup

Paired across the 10 repeats. Repeats reuse the same wines, so p-values are indicative.

3 · Every candidate: the accuracy vs. macro-F1 trade-off

Figure 5.7: Review 2 page: improving the result (part 1 of 2)



No model wins both metrics; the dashed line is the best-trade-off frontier

rank_by_macro_f1	model	kind	accuracy	balanced_accuracy	macro_f1
1	RF · SMOTE	single learner	67.94	42.07	40.92
2	Learned-T ensemble · SMOTE	ensemble	66.63	42.49	40.87
3	Paper ensemble · SMOTE	ensemble	63.86	41.44	39.34
4	Paper ensemble · class weights	ensemble	68.43	37.94	38.77
5	RF · no balancing	single learner	70.85	36.7	38.18
6	RF · Review-1 setup	single learner	71.03	36.4	38.09
7	RF · class weights	single learner	71.03	36.4	38.09
8	GradBoost · SMOTE	single learner	63.49	39.4	38.08
9	Learned-T ensemble · Review-1 setup	ensemble	70.89	36.58	38.08
10	Paper weights: SVM + RF + ExtraTrees + HistGB	ensemble	70.79	36.96	38.04
11	ExtraTrees	single learner	70.97	36.45	37.93

Figure 5.8: Review 2 page: improving the result (part 2 of 2)

Overview

Dataset

Review 1: reproduction

Review 2: class balancing

Review 2: improving the result

Feature importance

Predict a wine

Wine Quality Prediction

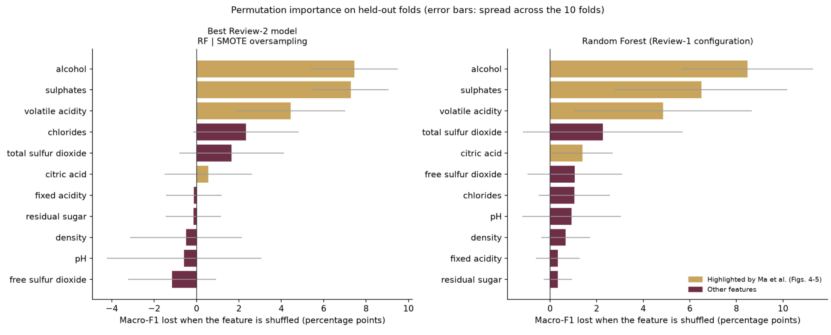
Surianandhan S · 127018060

CSE425 Machine Learning Essentials

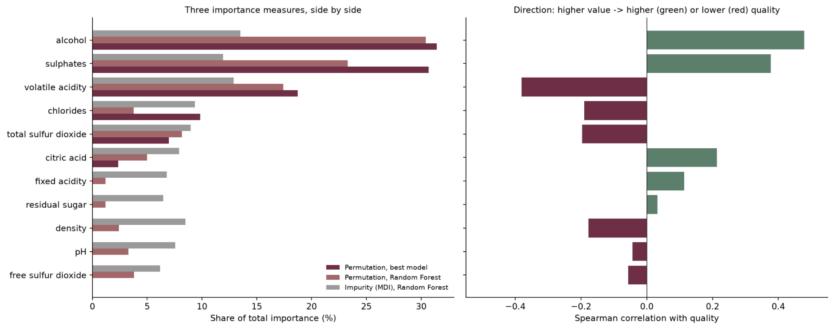
SASTRA Deemed to be University

Feature importance

Permutation importance on held-out folds (primary), Random Forest impurity importance and Spearman correlation, checked against each other and against the attributes the paper highlights.



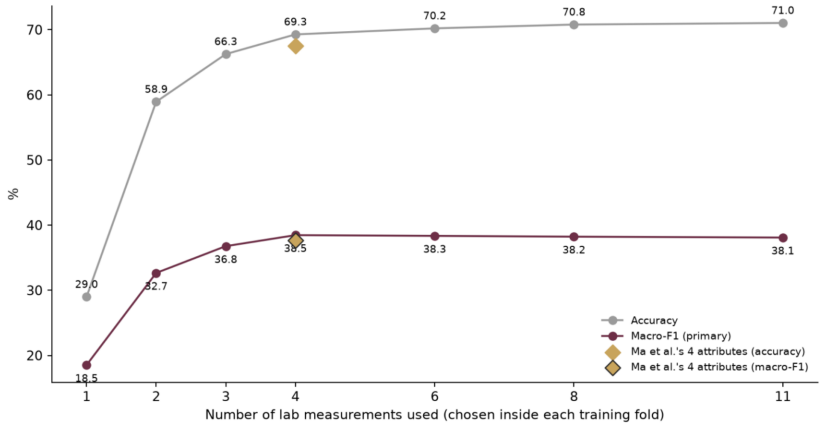
Macro-F1 lost when each measurement is shuffled



Three importance measures, and direction of effect

How many lab tests does the model need? (Random Forest, class-weighted)

Figure 5.9: Feature importance page (part 1 of 2)



How many lab tests the model needs

rank_final	feature	final_f1_drop	rf_f1_drop	rf_mdi	spearman_rho	highlighted_by_paper
1	alcohol	7.451	8.481	0.135	0.478	☑
2	sulphates	7.275	6.494	0.119	0.377	☑
3	volatile acidity	4.438	4.856	0.129	-0.381	☑
4	chlorides	2.331	1.046	0.094	-0.19	☐
5	total sulfur dioxide	1.656	2.273	0.09	-0.197	☐
6	citric acid	0.556	1.388	0.079	0.214	☑
7	fixed acidity	-0.128	0.336	0.068	0.114	☐
8	residual sugar	-0.149	0.336	0.064	0.032	☐
9	density	-0.498	0.672	0.085	-0.177	☐
10	pH	-0.586	0.919	0.076	-0.044	☐

Figure 5.10: Feature importance page (part 2 of 2)

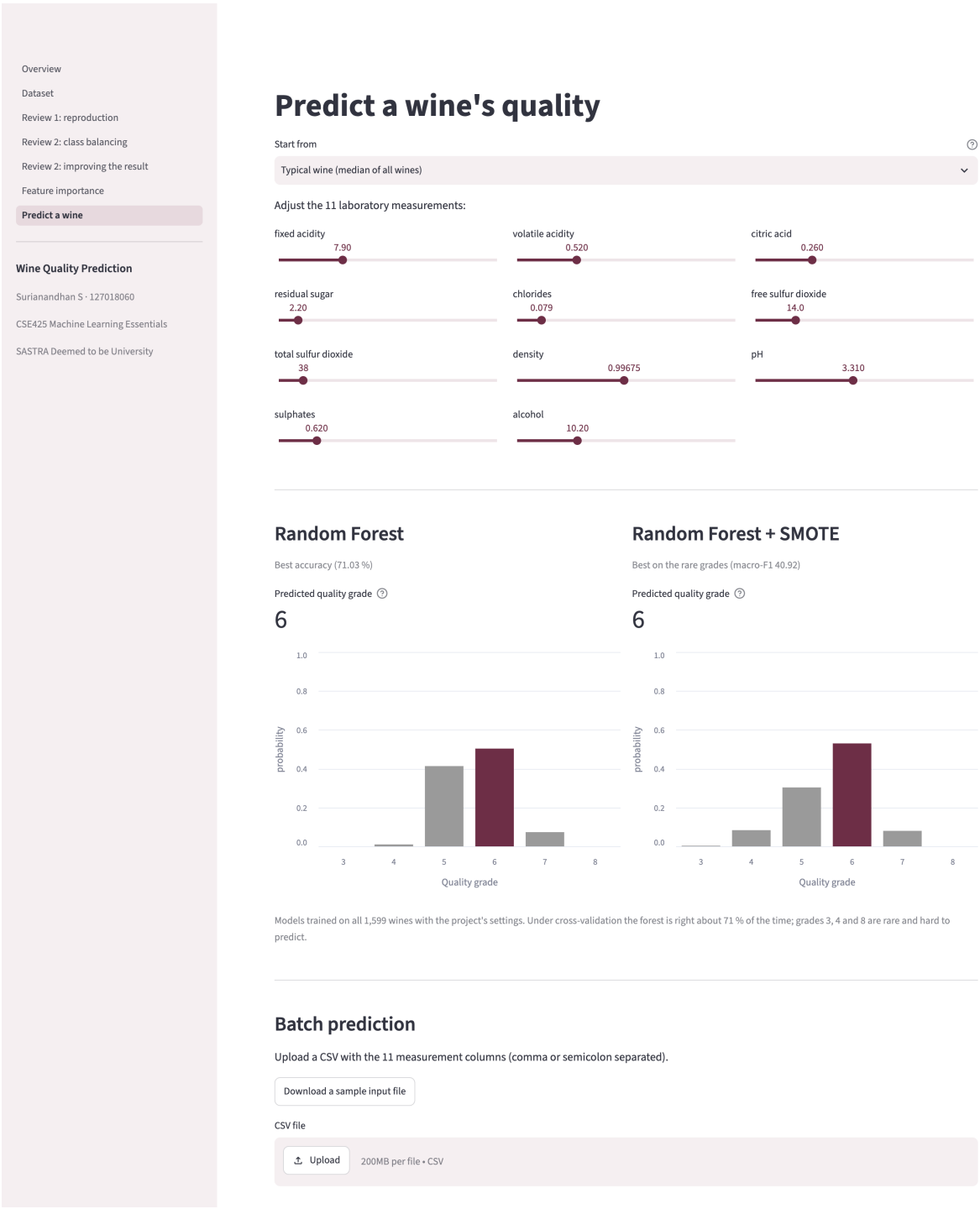


Figure 5.11: Predict page: grades and probabilities from both models, and batch prediction

CHAPTER 6
SOURCE CODE AND SNAPSHOTS

The complete source code is in the project repository (https://github.com/Surianandhan/wine-quality-ml-project). The excerpts below are taken directly from the project's files and show the core of each step; docstrings and comments are left out.

6.1 SETTINGS

```
RANDOM_STATE = 42
N_SPLITS = 10
N_REPEATS = 10
BENCHMARK_CLASSES = [3, 4, 5, 6, 7, 8]
MA_2024_OVERALL_ACCURACY = 89.73
REVIEW2_PRIMARY_METRIC = "macro_f1"
SMOTE_K_NEIGHBORS = 5
WEIGHT_TEMPERATURES = [1, 2, 5, 10, 25, 100]
FEATURE_SUBSET_SIZES = [1, 2, 3, 4, 6, 8, 11]
PERMUTATION_REPEATS = 20
```

6.2 DATA LOADING AND SCALING

```
def load_red_wine(path=DEFAULT_RED_WINE_PATH):
    return pd.read_csv(path, sep=";")

def get_features_target(df, feature_columns=None):
    feature_columns = feature_columns or FEATURE_COLUMNS
    X = df[feature_columns].to_numpy(dtype=float)
    y = df[TARGET_COLUMN].to_numpy(dtype=int)
    return X, y

def _pipe(estimator):
    return Pipeline([("scaler", MinMaxScaler()), ("clf", estimator)])
```

6.3 BASE LEARNERS

```
def get_base_learners(random_state=RANDOM_STATE):
    return {
        "SVM": _pipe(
            CalibratedClassifierCV(
                SVC(
                    kernel="rbf",
                    C=10.0,
                    gamma="scale",
                    class_weight="balanced",
                    random_state=random_state,
                ),
                ensemble=False,
            )
        ),
        "RF": _pipe(
            RandomForestClassifier(
                n_estimators=300,
                class_weight="balanced_subsample",
                random_state=random_state,
            )
        ),
        "MLP": _pipe(
            MLPClassifier(
                hidden_layer_sizes=(64, 32),
                max_iter=2000,
                random_state=random_state,
            )
        ),
```

```
        "GradBoost": _pipe(
            GradientBoostingClassifier(
                n_estimators=200,
                random_state=random_state,
            )
        ),
    }
```

6.4 ENSEMBLE WEIGHTS AND WEIGHTED VOTE

```
def _error_vector(y_true, y_pred):
    return (np.asarray(y_true) != np.asarray(y_pred)).astype(float)

def compute_weights(error_vectors):
    names = list(error_vectors.keys())
    cv_errors = {name: float(error_vectors[name].mean()) for name in names}  # Eq. (6)

    inv_err = {name: (1.0 / cv_errors[name] if cv_errors[name] > 0 else 1e6) for name in
names}
    total_inv = sum(inv_err.values())
    w_accuracy = {name: inv_err[name] / total_inv for name in names}

    D = {}
    for name in names:
        D[name] = sum(
            np.linalg.norm(error_vectors[name] - error_vectors[other])
            for other in names
            if other != name
        )
    total_D = sum(D.values())
    w_diversity = {name: D[name] / total_D for name in names}  # Eq. (9)

    w = {name: 0.5 * (w_accuracy[name] + w_diversity[name]) for name in names}  # Eq. (4)
    return w, w_accuracy, w_diversity, cv_errors

def weighted_soft_vote(probas, weights, classes):
    names = list(probas.keys())
    weighted_sum = np.zeros_like(next(iter(probas.values())), dtype=float)
    for name in names:
        weighted_sum += weights[name] * probas[name]
    classes_arr = np.asarray(classes)
    return classes_arr[weighted_sum.argmax(axis=1)]
```

6.5 REPEATED CROSS-VALIDATION

```
def run_repeated_cv(X, y, n_splits=N_SPLITS, n_repeats=N_REPEATS, base_seed=0):
    classes = sorted(np.unique(y).tolist())
    learners = get_base_learners()

    per_repeat = []
    weights_df_records = []
    pooled_true = []
    pooled_preds = {name: [] for name in learners}
    pooled_ensemble = []       # soft vote -- the headline ensemble
    pooled_ensemble_hard = []  # hard vote -- kept for comparison

    for r in range(n_repeats):
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=base_seed + r)

        fold_preds = {}
        fold_probas = {}
        for name, model in learners.items():
            proba = cross_val_predict(model, X, y, cv=skf, n_jobs=-1,
method="predict_proba")
            fold_probas[name] = _align_proba_columns(proba, classes, classes)
            fold_preds[name] = np.asarray(classes)[proba.argmax(axis=1)]

        error_vectors = {name: _error_vector(y, fold_preds[name]) for name in learners}
        w, w_acc, w_div, cv_err = compute_weights(error_vectors)
```

```
        ensemble_pred = weighted_soft_vote(fold_probas, w, classes)
        ensemble_pred_hard = weighted_vote(fold_preds, w, classes)

        accuracy = {name: float((fold_preds[name] == y).mean()) for name in learners}
        accuracy["Ensemble"] = float((ensemble_pred == y).mean())
        accuracy["Ensemble_hard_vote"] = float((ensemble_pred_hard == y).mean())

        ...  # per-repeat records are stored, then all repeats are pooled
```

6.6 CLASS BALANCING

```
class BalancedSampleWeight(ClassifierMixin, BaseEstimator):

    def __init__(self, estimator):
        self.estimator = estimator

    def fit(self, X, y):
        self.estimator_ = clone(self.estimator)
        self.estimator_.fit(X, y, sample_weight=compute_sample_weight("balanced", y))
        self.classes_ = self.estimator_.classes_
        return self

    def predict(self, X):
        return self.estimator_.predict(X)

    def predict_proba(self, X):
        return self.estimator_.predict_proba(X)

def make_paper_learners(balancing, random_state=RANDOM_STATE):
    learners = get_base_learners(random_state)
    if balancing == "partial":
        return learners

    if balancing in ("none", "smote"):
        learners["SVM"].set_params(clf__estimator__class_weight=None)
        learners["RF"].set_params(clf__class_weight=None)
    if balancing == "none":
        return learners

    if balancing == "class_weight":
        for name in ("MLP", "GradBoost"):

learners[name].set_params(clf=BalancedSampleWeight(learners[name].named_steps["clf"]))
        return learners

    if balancing == "smote":
        return {
            name: ImbPipeline(
                [
                    ("scaler", MinMaxScaler()),
                    ("smote", SMOTE(k_neighbors=SMOTE_K_NEIGHBORS,
random_state=random_state)),
                    ("clf", pipe.named_steps["clf"]),
                ]
            )
            for name, pipe in learners.items()
        }

    raise ValueError(f"unknown balancing method: {balancing!r}")
```

6.7 LEARNED TEMPERATURE

```
def tempered_weights(error_vectors, temperature=1, sharpen="both"):
    names = list(error_vectors)
    t_acc = temperature
    t_div = temperature if sharpen == "both" else 1

    cv_err = {n: float(error_vectors[n].mean()) for n in names}
```

```
    inv = {n: (1.0 / cv_err[n] if cv_err[n] > 0 else 1e6) ** t_acc for n in names}
    total_inv = sum(inv.values())
    w_acc = {n: inv[n] / total_inv for n in names}

    D = {
        n: sum(np.linalg.norm(error_vectors[n] - error_vectors[o]) for o in names if o !=
n)
        for n in names
    }
    Dt = {n: D[n] ** t_div for n in names}
    total_D = sum(Dt.values())
    w_div = {n: Dt[n] / total_D for n in names}

    w = {n: 0.5 * (w_acc[n] + w_div[n]) for n in names}
    if not all(np.isfinite(v) for v in w.values()):
        raise FloatingPointError(f"non-finite weights at T={temperature}")
    return w

class PaperWeightedEnsemble(ClassifierMixin, BaseEstimator):

    def __init__(self, members, temperatures=(1,), sharpen_modes=("both",),
                 selection_metric=REVIEW2_PRIMARY_METRIC, inner_splits=5,
random_state=RANDOM_STATE):
        self.members = members
        self.temperatures = temperatures
        self.sharpen_modes = sharpen_modes
        self.selection_metric = selection_metric
        self.inner_splits = inner_splits
        self.random_state = random_state

    def fit(self, X, y):
        self.classes_ = np.unique(y)
        skf = StratifiedKFold(n_splits=self.inner_splits, shuffle=True,
random_state=self.random_state)
        probas = {
            name: cross_val_predict(clone(model), X, y, cv=skf, method="predict_proba")
            for name, model in self.members.items()
        }
        errors = {n: _error_vector(y, self.classes_[p.argmax(axis=1)]) for n, p in
probas.items()}

        best = None
        for T in sorted(self.temperatures):
            for mode in self.sharpen_modes:
                w = tempered_weights(errors, T, mode)
                pred = weighted_soft_vote(probas, w, list(self.classes_))
                value = _metric(y, pred, self.selection_metric)
                if best is None or value > best[0] + 1e-12:
                    best = (value, T, mode, w)
        self.inner_score_, self.temperature_, self.sharpen_, self.weights_ = best
        self.fitted_ = {name: clone(model).fit(X, y) for name, model in
self.members.items()}
        return self

    def predict_proba(self, X):
        return sum(self.weights_[n] * m.predict_proba(X) for n, m in self.fitted_.items())

    def predict(self, X):
        return self.classes_[self.predict_proba(X).argmax(axis=1)]
```

6.8 FEATURE IMPORTANCE

```
def _one_fold(model, X, y, train, test, fold, seed):
    fitted = clone(model).fit(X[train], y[train])
    result = permutation_importance(
        fitted,
        X[test],
        y[test],
        scoring=["f1_macro", "accuracy"],
```

```
        n_repeats=PERMUTATION_REPEATS,
        random_state=seed + fold,
        n_jobs=1,
    )
    mdi = None
    clf = getattr(fitted, "named_steps", {}).get("clf")
    if clf is not None and hasattr(clf, "feature_importances_"):
        mdi = clf.feature_importances_
    return result["f1_macro"].importances_mean, result["accuracy"].importances_mean, mdi
```

6.9 WEB APPLICATION

```
def trained_models():
    X, y, _, _ = load_red_wine_xy()
    rf = clone(get_base_learners()["RF"]).fit(X, y)
    rf_smote = clone(make_paper_learners("smote")["RF"]).fit(X, y)
    return {list(MODELS)[0]: rf, list(MODELS)[1]: rf_smote}

def page_predict():
    st.title("Predict a wine's quality")
    df = load_data()
    models = trained_models()

    examples = {"Typical wine (median of all wines)": df[FEATURE_COLUMNS].median()}
    for g in (4, 5, 6, 7, 8):
        examples[f"Typical grade-{g} wine"] = df[df["quality"] == g]
[FEATURE_COLUMNS].median()
    start = st.selectbox("Start from", list(examples), help="Fills the sliders with the
median measurements of that group.")
    base = examples[start]

    st.write("Adjust the 11 laboratory measurements:")
    values = {}
    cols = st.columns(3)
    for i, f in enumerate(FEATURE_COLUMNS):
        lo, hi = float(df[f].min()), float(df[f].max())
        step = 10 ** np.floor(np.log10((hi - lo) / 200)) if hi > lo else 0.01
        decimals = int(min(5, max(0, -np.floor(np.log10(step)))))
        values[f] = cols[i % 3].slider(f, lo, hi, float(np.clip(base[f], lo, hi)),
step=float(step),
                                       format=f"%.{decimals}f", key=f"{start}-{f}")
    x = pd.DataFrame([values])[FEATURE_COLUMNS].to_numpy()

    st.divider()
    out = st.columns(len(models))
    for col, (name, model) in zip(out, models.items()):
        proba = model.predict_proba(x)[0]
        pred = int(model.classes_[proba.argmax()])
        with col:
            st.subheader(name)
            st.caption("Best accuracy (71.03 %)" if name == "Random Forest" else "Best on
the rare grades (macro-F1 40.92)")
            st.metric("Predicted quality grade", pred, help=MODELS[name])
            prob = pd.DataFrame({"grade": [str(c) for c in model.classes_], "probability":
proba})
            st.altair_chart(alt.Chart(prob).mark_bar().encode(
                x=alt.X("grade:O", title="Quality grade", axis=alt.Axis(labelAngle=0)),
```

6.10 OUTPUT SNAPSHOTS

The following snapshots show real outputs of the project: the automated checks of the web application and the headline comparison of Review 1 and Review 2 read from the result files. The figures of Chapters 2 to 4 are the graphical outputs of the experiments, and the web application is shown in Section 5.8.

```
$ python app/test_app.py
1. every page renders
  PASS  page renders: page_overview
  PASS  page renders: page_dataset
  PASS  page renders: page_reproduction
  PASS  page renders: page_balancing
  PASS  page renders: page_improvement
  PASS  page renders: page_importance
  PASS  page renders: page_predict
2. models match the project's settings
  PASS  Random Forest = Review-1 RF (identical predictions)
  PASS  RF + SMOTE = Review-2 SMOTE RF (identical predictions)
3. predictions are valid
  PASS  Random Forest: probabilities sum to 1
  PASS  Random Forest: predicted grades are real grades
  PASS  Random Forest + SMOTE: probabilities sum to 1
  PASS  Random Forest + SMOTE: predicted grades are real grades
4. headline numbers match the result files
  PASS  reproduction accuracy is 69.01
  PASS  best macro-F1 model is RF + SMOTE

ALL CHECKS PASSED
```

Figure 6.1: Output of the automated checks of the web application

```
$ python -c "import pandas as pd; print(pd.read_csv(
>   'results/review2/tables/headline_review1_vs_review2.csv', index_col=0).round(2))"
                  review1_ensemble  review1_random_forest  review2_best  change_vs_review1_ensemble
metric
accuracy                     69.01                  71.03         67.94                       -1.08
balanced_accuracy            35.52                  36.40         42.07                        6.55
macro_f1                     36.92                  38.09         40.92                        4.00
weighted_f1                  67.17                  69.12         68.22                        1.05
recall_3                      0.00                   0.00          4.00                        4.00
recall_4                      0.57                   0.00         21.89                       21.32
recall_5                     81.66                  81.97         78.00                       -3.66
recall_6                     69.87                  74.29         64.18                       -5.69
recall_7                     49.90                  51.06         65.43                       15.53
recall_8                     11.11                  11.11         18.89                        7.78
```

Figure 6.2: Output: headline metrics of Review 1 and the best Review-2 model

CHAPTER 7
MERITS AND DEMERITS

7.1 MERITS

1. **Faithful and leakage-free:** the paper's weighting equations are implemented as written, and every scaler, resampler, selector and setting is fitted inside the training folds of a 10 × 10 cross-validation.
2. **Checked against the paper's own numbers:** the gap to the published accuracy is explained with checks computed from the paper's tables, reported as figures that could not be reconciled rather than as accusations.
3. **Honest about trade-offs:** macro-F1 was fixed as the Review-2 metric before the experiments, accuracy is always reported beside it, and both the most accurate and the most balanced model are offered.
4. **Robust conclusions:** the main comparisons are confirmed repeat by repeat, and three different importance measures agree.
5. **Reproducible and usable:** one command reruns every experiment, the numbers in the report are read from the result files, and a tested web application serves the models.

7.2 DEMERITS

1. **Two learners are substitutes:** Gradient Boosting and the MLP replace the RNN and the ELM, so the reproduction is not exact for those two members.
2. **Rare grades remain hard:** with 10 wines of grade 3 and 18 of grade 8, even the best model recognises few of them, and the estimates for those grades are uncertain.
3. **Only red wine:** the white-wine data of the same source are not used, so the findings are not tested on a second dataset.
4. **Optimistic statistics:** the best model is chosen on the same cross-validation that scores it, and the repeats are not independent, so the paired p-values are indicative only.
5. **Physicochemical inputs only:** price, region, vintage and taster information are not available, which limits any predictor of a sensory grade.

CHAPTER 8

CONCLUSION AND FUTURE PLANS

This project reproduced the accuracy- and diversity-weighted ensemble of Ma et al. for predicting the quality of red wine from its physicochemical measurements, and extended it with class balancing, an improved ensemble and a study of feature importance. Evaluated with stratified 10-fold cross-validation repeated 10 times and without leakage, the reproduced ensemble reaches 69.01 % accuracy against the published 89.73 %, and Random Forest alone 71.03 %. The gap is explained by the paper's own tables: its weights are close to uniform, its base accuracies imply an ensemble near 67.52 %, and its confusion matrix implies a 491-wine hold-out with 82.08 % accuracy.

The second review's requests were completed. Class balancing with SMOTE inside the training folds raised macro-F1 from 36.92 to 40.92 and grade-4 recall from 0.57 % to 21.89 %, at a stated cost of 1.08 accuracy points. Learning the sharpness of the weights inside each training fold improved the paper's ensemble to 70.89 % in every repeat. Alcohol, sulphates and volatile acidity drive the prediction, and four routine tests give nearly all of the accuracy. The results are delivered through a tested web application.

Future Work:

1. Repeat the study on the white-wine data and on both colours together.
2. Treat quality as an ordered target (ordinal regression), so that predicting 6 for a grade-7 wine costs less than predicting 4.
3. Use a nested cross-validation to select the final model and a corrected resampled t-test for confidence intervals.
4. Try cost-sensitive thresholds that let a retailer choose how many unusual wines to flag.
5. Extend the web application with explanations of individual predictions.

CHAPTER 9
REFERENCES

1. Bhardwaj, P., Tiwari, P., Olejar, K., Parr, W., & Kulasiri, D. (2022). A machine learning application in wine quality prediction. *Machine Learning with Applications, 8*, 100261. https://doi.org/10.1016/j.mlwa.2022.100261
2. Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324
3. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953
4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018
5. Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems, 47*(4), 547–553. https://doi.org/10.1016/j.dss.2009.05.016
6. Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Wine Quality* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C56S3T
7. Dahal, K. R., Dahal, J. N., Banjade, H., & Gaire, S. (2021). Prediction of wine quality using machine learning algorithms. *Open Journal of Statistics, 11*(2), 278–289. https://doi.org/10.4236/ojs.2021.112015
8. Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30. https://jmlr.org/papers/v7/demsar06a.html
9. Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS 2000), Lecture Notes in Computer Science, Vol. 1857* (pp. 1–15). Springer. https://doi.org/10.1007/3-540-45014-9_1
10. Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research, 20*(177), 1–81. https://jmlr.org/papers/v20/18-760.html
11. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451
12. Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning, 63*(1), 3–42. https://doi.org/10.1007/s10994-006-6226-1

13. Gupta, Y. (2018). Selection of important features and predicting wine quality using machine learning techniques. *Procedia Computer Science, 125*, 305–312. https://doi.org/10.1016/j.procs.2017.12.041

14. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

15. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

16. Huang, G.-B., Zhu, Q.-Y., & Siew, C.-K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing, 70*(1–3), 489–501. https://doi.org/10.1016/j.neucom.2005.12.126

17. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

18. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates.

19. Kuncheva, L. I., & Whitaker, C. J. (2003). Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning, 51*(2), 181–207. https://doi.org/10.1023/A:1022859003006

20. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research, 18*(17), 1–5. https://jmlr.org/papers/v18/16-365.html

21. Ma, R., Mao, D., Cao, D., Luo, S., Gupta, S., & Wang, Y. (2024). From vineyard to table: Uncovering wine quality for sales management through machine learning. *Journal of Business Research, 176*, 114576. https://doi.org/10.1016/j.jbusres.2024.114576

22. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

23. Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*(3), 239–281. https://doi.org/10.1023/A:1024068626366

24. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

25. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536. https://doi.org/10.1038/323533a0

26. Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

27. Streamlit Inc. (2026). *Streamlit* (Version 1.66.0) [Computer software]. https://streamlit.io

28. Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, 25. https://doi.org/10.1186/1471-2105-8-25

29. VanderPlas, J., Granger, B., Heer, J., Moritz, D., Wongsuphasawat, K., Satyanarayan, A., Lees, E., Timofeev, I., Welsh, B., & Sievert, S. (2018). Altair: Interactive statistical visualizations for Python. *Journal of Open Source Software, 3*(32), 1057. https://doi.org/10.21105/joss.01057

30. Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... van Mulbregt, P. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

31. Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin, 1*(6), 80–83. https://doi.org/10.2307/3001968

32. Wolpert, D. H. (1992). Stacked generalization. *Neural Networks, 5*(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

33. Zhou, Z.-H. (2012). *Ensemble methods: Foundations and algorithms*. Chapman & Hall/CRC. https://doi.org/10.1201/b12207